GATE CSE

Algorithms

The Complete Beginner-to-Exam Guide



Complexity • Recurrences • Divide & Conquer • Graphs • Shortest Paths • MST • Hashing • Greedy • Dynamic Programming • Recursion • Heaps & BST

12 chapters • worked examples • shortcuts • traps • PYQ map

GATE CSE
Algorithms

The Complete Beginner-to-Exam Guide

Contents

Read This First 1

0.1 How much is Algorithms worth in GATE? . . . 2
0.2 Question styles you will actually see . . . 2
0.3 Symbols and conventions used in this book . . . 3
0.4 Suggested 3-week plan . . . 3

Part I • Foundations . . . 4

1 Measuring Algorithms: Complexity and Asymptotic Notation 5

1.1 Starting from absolute zero . . . 5
1.2 Best, worst and average case . . . 5
1.3 Asymptotic notation — the three symbols . . . 6
1.4 The master growth ladder (memorise!) . . . 6
1.5 How to compare two weird functions (3 methods) . . . 7
1.6 Counting steps in loops (the "code complexity" PYQ type) . . . 8
1.7 Practice — Chapter 1 . . . 9

2 Solving Recurrences 10

2.1 What is a recurrence? . . . 10
2.2 Method 1 — Repeated substitution (unrolling) . . . 10
2.3 Method 2 — Recursion Tree (for unequal splits) . . . 11
2.4 Method 3 — Master Theorem (the workhorse) . . . 13
2.5 Extended (Complete) Master Theorem . . . 14
2.6 Method 4 — Change of variable (for $\sqrt{n}$ and similar) . . . 15
2.7 Master flow-chart (print this) . . . 16
2.8 Practice — Chapter 2 (do all before moving on) . . . 16

Part II • Divide & Conquer and Sorting . . . 18

3 Divide & Conquer, Merge Sort, Quick Sort and the Sorting Family 19

3.1 The D&C idea . . . 19
3.2 Maximum of an array (the "hello world" of D&C) . . . 19
3.3 Merge Sort . . . 19
3.4 Merge-sort variants asked in your notes . . . 21
3.5 Merging K sorted arrays (total n elements, each n/K long) . . . 22
3.6 Stable and in-place sorting . . . 22
3.7 Maximum, minimum and second maximum — counting comparisons . . . 23
3.8 Counting inversions . . . 24
3.9 Closest pair of points in 2-D . . . 25
3.10 Fast exponentiation a^b . . . 25
3.11 Matrix multiplication . . . 25
3.12 Quick Sort . . . 26

3.13 The Selection problem (k-th smallest) . . . 27
3.14 Binary search . . . 27
3.15 The sorting algorithms in detail . . . 28
3.16 Practice — Chapter 3 . . . 29

Part III • Graphs . . . 30

4 Graph Basics, DFS and BFS 31
4.1 Graph vocabulary from scratch . . . 31
4.2 How a graph is stored . . . 32
4.3 Depth-First Search (DFS) . . . 32
4.4 Edge classification . . . 34
4.5 DFS application 1 — Cycle detection . . . 35
4.6 DFS application 2 — Topological sort . . . 35
4.7 DFS application 3 — Articulation points (cut vertices) . . . 36
4.8 Strongly Connected Components (SCC) . . . 38
4.9 Breadth-First Search (BFS) . . . 38
4.10 DFS vs BFS cheat-sheet . . . 40
4.11 Practice — Chapter 4 . . . 41

5 Shortest Paths: Greedy Thinking, Dijkstra, Bellman-Ford, Floyd-Warshall 42
5.1 What is a "greedy" algorithm? . . . 42
5.2 The Single-Source Shortest Path (SSSP) problem . . . 42
5.3 Dijkstra's algorithm (non-negative weights) . . . 43
5.4 Shortest path in a DAG (works with negative edges!) . . . 46
5.5 Bellman–Ford algorithm (negative edges allowed) . . . 47
5.6 All-pairs shortest paths: Floyd–Warshall . . . 48
5.7 Travelling Salesman (TSP) . . . 49
5.8 Shortest-path cheat-sheet . . . 49
5.9 Practice — Chapter 5 . . . 49

6 Minimum Spanning Trees 51
6.1 What is a spanning tree? . . . 51
6.2 The two golden properties . . . 51
6.3 Kruskal's algorithm . . . 52
6.4 Prim's algorithm . . . 53
6.5 Prim vs Dijkstra, and MST ≠ shortest-path tree . . . 54
6.6 Counting MSTs (the NAT favourite) . . . 55
6.7 MST properties — true/false statements . . . 56
6.8 "Missing weight / range of x" problems . . . 56
6.9 Maintaining an MST when one edge weight changes . . . 57
6.10 Practice — Chapter 6 . . . 57

Part IV • Hashing . . . 58

7 Hashing 59
7.1 The problem hashing solves . . . 59
7.2 Collision resolution method 1 — Chaining (separate chaining) . . . 59
7.3 Collision resolution method 2 — Open addressing . . . 60
7.4 Fully traced examples (the NAT pattern) . . . 61
7.5 Analysis under the Simple Uniform Hashing Assumption (SUHA) . . . 62
7.6 Chaining vs open addressing . . . 62

7.7 Practice — Chapter 7 . . . 63

Part V • Greedy Algorithms and Dynamic Programming . . . 64

8 Greedy (Huffman, Scheduling, Knapsack) and Dynamic Programming 65
8.1 Huffman Encoding . . . 67
8.2 Optimal Merge Pattern . . . 68
8.3 Interval (Activity) Scheduling . . . 69
8.4 Job Sequencing with Deadlines . . . 70
8.5 Fractional Knapsack . . . 70
8.6 Greedy cheat-sheet . . . 70
8.7 Why DP? . . . 72
8.8 Longest Common Subsequence (LCS) . . . 72
8.9 Matrix Chain Multiplication (MCM) . . . 73
8.10 0/1 Knapsack . . . 75
8.11 Other DP patterns worth knowing (one-line recurrences) . . . 76
8.12 Greedy vs Dynamic Programming — decision rule . . . 76
8.13 Practice — Chapter 8 . . . 76

Part VI • Recursion, Heaps, Trees, Searching . . . 78

9 Recursion and Code Tracing 79
9.1 How recursion works (from zero) . . . 79
9.2 The 5-step tracing method . . . 79
9.3 Pattern library (with verified answers) . . . 80
9.4 Worked examples . . . 80
9.5 Time & space of recursive code — fast verdicts . . . 81
9.6 Practice — Chapter 9 . . . 81

10 Heaps, Binary Search Trees and Searching 82
10.1 Binary heap . . . 82
10.2 Binary Search Tree (BST) . . . 84
10.3 Searching . . . 84
10.4 Practice — Chapter 10 . . . 85

Part VII • Revision Tools . . . 86

11 One-Page Cheat Sheets 87
11.1 Complexity master table . . . 87
11.2 Recurrence one-liners . . . 88
11.3 Graph algorithm chooser . . . 89
11.4 Top 40 facts to know cold . . . 89
11.5 Exam-day strategy . . . 90

12 PYQ Practice Map — Solve These Yourself 91
12.1 Suggested sequence for PYQ practice . . . 92
12.2 Pattern → what to do (from the cluster analysis) . . . 92

Appendix A: What came from your notes vs what I added 93

Read This First

Who is this for? Someone opening Algorithms for the very first time who wants to reach "I can solve GATE questions in 2 minutes". **What it replaces:** the long video lectures. Every topic, every worked example, and every shortcut from the lecture notes is here — rewritten in plain language — plus extra depth on the topics your PYQ analysis says GATE loves. **How to use it:** read a chapter → redo every worked example on paper without looking → do the *Practice* set → then go solve the real PYQs listed in Chapter 12.

0.1 How much is Algorithms worth in GATE?

From your PYQ analysis (GATE 2018–2026): Algorithms averages **9.1 marks/paper** (range 6–14; **GATE 2026 had 14**). The top 3 clusters alone are ~57% of the subject's marks.

Priority	Topic cluster	Avg marks/paper	Years asked ('18--'26)	Book chapter
🔴 **Must-do**	Asymptotic notation, time complexity & recurrences	**1.61**	8 / 9	Ch 1–2
🟠 **Next**	Graph traversal & shortest paths (BFS / DFS / Dijkstra / Bellman-Ford / SCC / Topo-sort)	**2.06** (highest!)	7 / 9	Ch 4–5
🟠 **Next**	Minimum Spanning Tree (Kruskal / Prim)	**1.56**	7 / 9	Ch 6
🟠 **Next**	Hashing (open addressing, probing)	0.67	6 / 9	Ch 7
🟡 If time permits	Recursion & code tracing ("what does this compute?")	1.06	5 / 9	Ch 9
🟡 If time permits	Dynamic programming, greedy & Huffman	1.00	5 / 9	Ch 8
🟡 If time permits	Sorting (quick / merge / heap, comparisons, lower bounds)	0.50	5 / 9	Ch 3
⚪ Skim	Searching, BST, heaps in algorithms	0.67 (but 4 marks in 2026!)	3 / 9	Ch 10

Why the order of chapters follows weight, not the lecturer's order: Ch 1–2 (complexity + recurrences) is the foundation of *everything else* and the single most reliable scoring area. Graphs and MST are the 2-mark question factories.

0.2 Question styles you will actually see

- **MCQ** – one correct option. **MSQ** – one or more correct; *no partial marks if you pick a wrong one*, so be sure of each statement. **NAT** – type a number; no options to eliminate, so *practise computing exactly*.
- Typical asks: solve a recurrence · order growth rates · count MSTs / MST weight · "which statements about BFS/DFS are true" · trace Dijkstra/Bellman-Ford · fill a hash table · trace a recursive function · DP recurrence · Huffman code lengths.

0.3 Symbols and conventions used in this book

- log n means log base 2 unless stated. lg = $\log_2$, ln = natural log. Base changes only multiply by a constant, so inside $\Theta(\ )$ the base does not matter.
- $|\mathbf{V}|$ = number of vertices, $|\mathbf{E}|$ = number of edges. Written **V** and **E** inside complexities.
- 📝 **Notes p. X** → this example is on page X of your lecture-notes PDF (spread k = pages 2k−1 and 2k).
- ⚡ **GATE Shortcut** → time-saving rule.
- ⚠️ **Trap** → the mistake examiners are waiting for.
- 🆕 **Added** → topic not in your lecture notes but important per the PYQ analysis, so I wrote it from scratch.

0.4 Suggested 3-week plan

Days	Do
1–3	Ch 1–2 (complexity, recurrences) — do *all* practice
4–7	Ch 4–5 (graphs, shortest paths)
8–10	Ch 6 (MST) — counting problems are pure practice
11–12	Ch 7 (hashing) + Ch 3 (D&C, sorting)
13–15	Ch 8 (DP, greedy, Huffman)
16–17	Ch 9–10 (tracing, heaps/BST)
18–21	Ch 12: solve every listed PYQ; revisit Ch 11 cheat-sheet daily

PART

I

Foundations

🔴 Must-do

IN THIS PART

Ch 1 Measuring Algorithms: Complexity and Asymptotic Notation

Ch 2 Solving Recurrences

CHAPTER 1

Measuring Algorithms: Complexity and Asymptotic Notation

📝 Notes p. 1 (introduction). Everything below goes well beyond that one page because this is the most-tested area.

1.1 Starting from absolute zero

An **algorithm** is a step-by-step recipe that turns an input into an output. Two algorithms can solve the same problem, but one may take 1 second and the other 1 year. We need a fair way to compare them.

An algorithm is judged on two things:

1. **Time complexity** — *how the running time grows as the input size grows.* (Notes: "the order of magnitude of time it takes to run an algorithm as a function of its input size".)
2. **Space complexity** — *how much extra memory it uses as a function of input size.*

> We never measure in seconds (that depends on your laptop). We count **basic steps** (comparisons, assignments) and see how that count grows with **n**, the input size.

1.2 Best, worst and average case

The same algorithm can behave differently on different inputs of the *same size*:

- **Best case** – the luckiest input (e.g. array already sorted for insertion sort).
- **Worst case** – the unluckiest input (reverse sorted). **GATE asks worst case unless told otherwise.**
- **Average case** – expected cost over all inputs.

For each case you may report the cost using **O, Ω or Θ** — and this is where students get confused, so read the next section slowly.

> ⚠️ **Trap:** "Best/worst case" and "O/Ω/Θ" are **independent ideas**. *Best/worst case* picks which **input**. $O/\Omega/\Theta$ says whether you are giving an upper, lower or tight **bound** on the cost of that case. You can say "the worst case of insertion sort is $\Theta(n^2)$" or "the best case is $\Omega(n)$".

1.3 Asymptotic notation — the three symbols

We ignore constants and small terms because for large n only the fastest-growing term matters. ($3n^2 + 100n + 7$ behaves like n^2.)

Symbol	Meaning (intuition)	Formal definition	Think of it as
$O(g)$	f grows **no faster than** g (upper bound)	$\exists\, c>0, n_0 : f(n) \le c\cdot g(n)\ \forall\ n \ge n_0$	$f \le g$
$\Omega(g)$	f grows **at least as fast as** g (lower bound)	$\exists\, c>0, n_0 : f(n) \ge c\cdot g(n)\ \forall\ n \ge n_0$	$f \ge g$
$\Theta(g)$	f grows **exactly like** g (tight)	$f = O(g)$ **and** $f = \Omega(g)$	$f = g$
$o(g)$	f grows **strictly slower** than g	$\lim f/g = 0$	$f < g$
$\omega(g)$	f grows **strictly faster** than g	$\lim f/g = \infty$	$f > g$

The limit trick (fastest method): compute $\lim f(n)/g(n)$ as $n\to\infty$.

- limit $= 0 \to f = o(g)$ (so $f = O(g)$, not Ω).
- limit = a finite positive constant $\to f = \Theta(g)$.
- limit $= \infty \to f = \omega(g)$ (so $f = \Omega(g)$, not O).

> ⚡**GATE Shortcut:** *O is a ceiling, Ω is a floor, Θ is both.* A loose ceiling is still true: $n = O(n^2)$ and $n = O(2^n)$ are both **true** statements (but not tight). Many MSQ options exploit this.

1.4 The master growth ladder (memorise!)

From slowest to fastest:

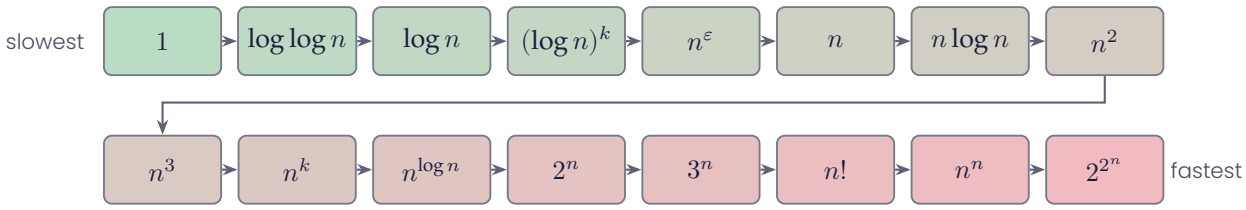


Figure 1.1. The master growth ladder, from slowest-growing to fastest-growing (read left to right, then continue on the second row).

Some less obvious positions:

- $(\log n)^{\log n} = n^{\log\log n} \to$ bigger than every polynomial, smaller than $n^{\log n}$.
- $n^{\log n} = 2^{\log^2 n} \to$ smaller than $2^{\sqrt{n}}$, which is smaller than 2^n.
- $\log(n!) = \Theta(n \log n)$ (Stirling), but $n!$ itself is much bigger than 2^n.
- $n^{1/\log n} = 2 \to$ this is just a **constant**, $\Theta(1)$.
- $2^{n+1} = \Theta(2^n)$ (constant factor 2) **but** $2^{2n} = 4^n \neq O(2^n)$.
- $(n+k)^m = \Theta(n^m)$ for constants k, m.

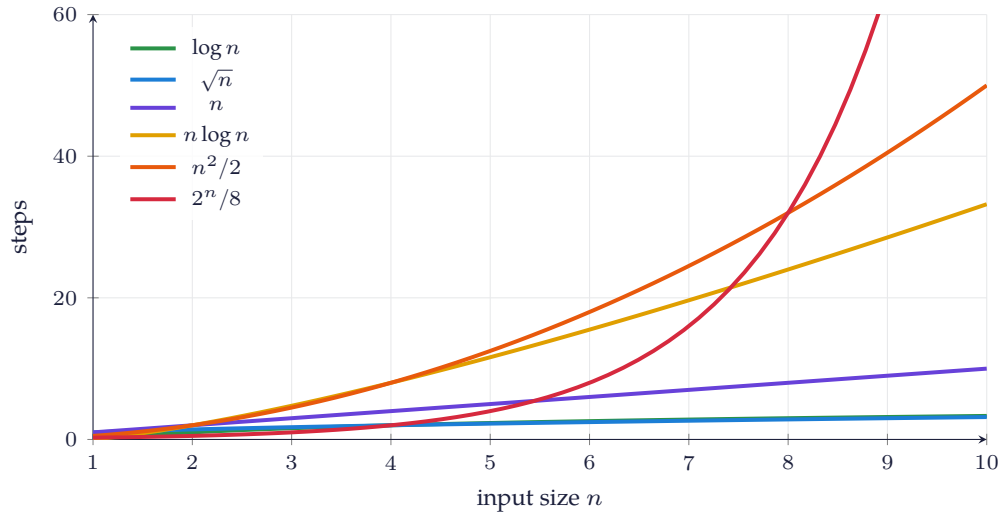


Figure 1.2. Growth rates at a glance: the exponential curve leaves every polynomial behind very quickly (constants scaled only to keep the plot readable).

1.5 How to compare two weird functions (3 methods)

1. **Take logs of both** (works when both are positive & increasing). Compare $\log f$ and $\log g$. *Example:* $n^{\log n}$ vs 2^n: logs are $\log^2 n$ vs $n \to n$ wins $\to 2^n$ is bigger.
2. **Substitute a big number** (n = 2^20, say) — rough but fast for MCQ.
3. **Divide and take the limit** (L'Hôpital if needed).

> ⚠️ **Trap:** Taking logs is only valid for *ordering*; to claim Θ-equality, the logs must differ by an **additive constant**, not just be Θ of each other. (2^{2n} and 2^n have logs $2n$ and n — Θ-equal, but the original functions are not.)

1.6 Counting steps in loops (the ``code complexity'' PYQ type)

Rule: **multiply** nested independent loops; **add** sequential blocks; the **variable's update rule** decides the iteration count.

Loop header	# iterations	Why
`for(i=1;i<=n;i++)`	n	step +1
`for(i=1;i<=n;i+=k)` (k const)	$n/k = \Theta(n)$	
`for(i=1;i<n;i*=2)`	log n	doubles each time
`for(i=n;i>1;i/=2)`	log n	halves each time
`i=2; while(i<n) i=i*i`	log log n	i = 2, 4, 16, 256 ...
`for(i=0;i*i<n;i++)`	$\sqrt{n}$	stops when $i^2 \geq n$
`for(i=1;i<=n;i++) for(j=1;j<=i;j++)`	$n(n+1)/2 = \Theta(n^2)$	triangular sum
`for(i=1;i<=n;i++)` `for(j=1;j<=n;j*=2)`	n log n	inner is log n
`for(i=1;i<=n;i++)` `for(j=1;j<=n;j+=i)`	$n(1+½+⅓+\ldots) =$ **n ln n** $= \Theta(n \log n)$	harmonic series
`for(i=1;i<n;i*=2) for(j=0;j<i;j++)`	$1+2+4+\ldots+n/2 = \Theta(n)$	geometric sum
`for(i=n;i>0;i/=2) for(j=0;j<i;j++)`	$n + n/2 + \ldots = 2n = \Theta(n)$	geometric sum
triple nest $i \leq n, j \leq i, k \leq j$	$n^3/6 = \Theta(n^3)$	

⚡ **GATE Shortcut (the harmonic trap):** when the inner step is the **outer variable** ($j+=i$), the total is $n \cdot H(n) = \Theta(n \log n)$, **not** n^2. When the inner **limit** is the outer variable ($j \leq i$), it is $\Theta(n^2)$.

⚡ **Geometric-vs-arithmetic:** if the inner loop runs $n, n/2, n/4 \ldots$ or $1, 2, 4, \ldots$ you are summing a GP and the answer is dominated by the biggest term → $\Theta(n)$ — not n log n.

1.7 Practice — Chapter 1

🎯 TRY IT YOURSELF -- cover the answers first

Q1. Arrange in increasing order of growth: n^2, 2^n, $n \log n$, $\log(n!)$, $n^{\log n}$, $\sqrt{n}$, $\log^2 n$, $n!$. **A1.** `log² n < √n < {n log n ≡ log(n!)} < n² < n^(log n) < 2ⁿ < n!`

Q2. True/False: (a) 2^(n+1) = O(2ⁿ) (b) 2^(2n) = O(2ⁿ) (c) 3ⁿ = O(2ⁿ) (d) log(n!) = Θ(n log n) (e) n^(1/log n) = Θ(1). **A2.** (a) **T** (b) **F** (c) **F** (d) **T** (e) **T** (it equals 2).

Q3. $\sqrt{\log n}$ vs $\log \log n$ — which grows faster? **A3.** Put x = log n: $\sqrt{x}$ vs $\log x \to \sqrt{x}$ is faster. So $\sqrt{(\log n)} = \omega(\log \log n)$.

Q4. Complexity of

PSEUDOCODE

```
for (i = 1; i <= n; i++)
    for (j = 1; j <= n; j += i)
        count++;
```

A4. Θ(n log n) (harmonic). Quick check n=8,16,32 gives 24, 61, 145 steps — grows slightly faster than linear.

Q5. Complexity of `for(i=1;i<n;i*=2) for(j=0;j<i;j++) x++;` **A5.** 1+2+4+...+n/2 ≈ n → **Θ(n)**.

Q6. `f(n) = Σ_{i=1..n} i` and `g(n) = Σ_{i=1..n} i²` — relation? **A6.** f = n²/2 = Θ(n²), g = n³/3 = Θ(n³) → f = o(g).

Q7. (MSQ-style) If f(n) = O(g(n)) and g(n) = O(h(n)), which must be true? (a) f = O(h) (b) f = Θ(h) (c) h = Ω(f). **A7.** (a) and (c) true (transitivity); (b) false.

CHAPTER 2

Solving Recurrences

the #1 scoring skill

📝 Notes pp. 2–16. GATE asked "solve a recurrence" **8 times** in 2020–2026 (5×1M + 3×2M). Learn every method below.

2.1 What is a recurrence?

Recursive algorithms call themselves on smaller inputs. Their running time is described by an equation about **itself**:

```
T(n) = (number of sub-problems) · T(size of each) + (work to split & combine)
```

Merge sort: $T(n) = 2T(n/2) + n$ — "two half-size problems, plus n work to merge". Our goal: turn this into a $\Theta(\ )$ expression with no T on the right.

There are four tools. Pick using this quick guide:

Recurrence looks like…	Best tool
$T(n) = T(n-1) + f(n)$ or $T(n-k)$ …	Repeated substitution (unrolling)
`T(n) = aT(n/b) + f(n)`	**Master theorem** (basic or extended)
unequal pieces $T(n/3) + T(2n/3) + \ldots$	**Recursion tree** with lower/upper bound
$T(\sqrt{n})$, `T(n^{1/3})`	**Change of variable** n = 2^m

2.2 Method 1 — Repeated substitution (unrolling)

📝 Lecture notes pp. 2–3

Keep replacing T on the right using the definition until you reach the base case, spot the pattern after k steps, then set the argument = base.

Ex 1. $T(n) = T(n-1) + 1, T(0) = 1$

```
T(n) = T(n-1) + 1 = T(n-2) + 2 = T(n-3) + 3 = … = T(n-k) + k
Base: n - k = 0 → k = n   ⇒   T(n) = T(0) + n = n + 1   ⇒ Θ(n)
```

Ex 2. $T(n) = T(n-1) + n, T(1) = 1$

```
T(n) = n + (n-1) + (n-2) + … + 2 + 1 = n(n+1)/2   ⇒ θ(n²)
```

Ex 3. $T(n) = 2T(n/2) + n \log n, T(1) = 1$

```
Unroll k times:  T(n) = 2^k T(n/2^k) + n[ log n + log(n/2) + log(n/4) + … + log(n/2^(k-1)) ]
Stop when n/2^k = 1 → k = log n.
Bracket = Σ_{i=0}^{k-1} (log n - i) = k·log n - k(k-1)/2 = log²n - log n(log n -1)/2 = (log²n + log
↪  n)/2
T(n) = n + n(log²n + log n)/2        ⇒ θ(n log² n)
```

> ⚡ **Shortcut for "T(n)=2T(n/2)+n log n":** result is $n \log^2 n$ — one extra log factor over merge sort. (This is exactly the Extended Master case you meet in 2.6.)

Ex 4 (standard). Common patterns worth memorising:

Recurrence	Solution
$T(n) = T(n-1) + 1$	$\Theta(n)$
$T(n) = T(n-1) + n$	$\Theta(n^2)$
$T(n) = T(n-1) + \log n$	$\Theta(n \log n)$ (since $\Sigma \log i = \log n!$)
$T(n) = T(n-1) + 1/n$	$\Theta(\log n)$ (harmonic)
$T(n) = 2T(n-1) + 1$	$\Theta(2^n)$ (Tower of Hanoi: $2^n - 1$)
$T(n) = 2T(n-1) + n$	$\Theta(2^n)$
$T(n) = T(n-1) + T(n-2) + 1$	$\Theta(\varphi^n) \approx 1.618^n$, bounded by $\Omega(2^{n/2})$, $O(2^n)$
$T(n) = T(n/2) + 1$	$\Theta(\log n)$ (binary search)
$T(n) = T(n/2) + n$	$\Theta(n)$
$T(n) = 2T(n/2) + 1$	$\Theta(n)$
$T(n) = 2T(n/2) + n$	$\Theta(n \log n)$
$T(n) = 2T(n/2) + n^2$	$\Theta(n^2)$

2.3 Method 2 — Recursion Tree (for unequal splits)

📝 Lecture notes pp. 3–5

Draw the recursion as a tree, write the **cost of each level**, and add the levels.

Key facts

- If a **full level** costs the same (say n) then total = (cost per level) × (number of levels).
- With **unequal branches** the tree is lopsided: the shortest root-to-leaf path gives a **lower bound** on the number of levels, the longest gives an **upper bound**. If both bounds give the same Θ you are done.

Ex A. $T(n) = T(n/5) + T(4n/5) + n$

```
Level cost: n/5 + 4n/5 = n  at every full level.
Shortest branch (÷5 each time):   depth = log₅ n   → T(n) ≥ n·log₅ n
Longest  branch (×4/5 each time): depth = log_{5/4} n → T(n) ≤ n·log_{5/4} n
Both are θ(n log n)   ⇒  T(n) = θ(n log n)
```

Ex B. $T(n) = T(n/3) + T(2n/3) + n$ — same story: LB $n \log_3 n$, UB `n log_{3/2} n` ⇒ **Θ(n log n)**.

> ⚡**GATE Shortcut (the 'fractions add to 1' rule):** For $T(n) = \Sigma T(\alpha_i n) + n$, look at **Σ αᵢ**:
>
> - **Σ αᵢ < 1** → root dominates → **Θ(n)** (e.g. $T(n/2) + T(n/4) + T(n/8) + n \to 7/8 < 1 \to \Theta(n)$)
> - **Σ αᵢ = 1** → every level costs n → **Θ(n log n)** (Ex A, Ex B)
> - **Σ αᵢ > 1** → leaves dominate → polynomially bigger than n.

Ex C. $T(n) = T(n/3) + T(2n/3) + n^2$

```
Level 0: n²
Level 1: (n/3)² + (2n/3)² = (1/9 + 4/9) n² = (5/9) n²
Level k: (5/9)^k n²   — a DECREASING geometric series
Sum = n²[1 + 5/9 + (5/9)² + …] = n² · (constant < 9/4)   ⇒ θ(n²)
```

(Lower bound is the root alone, n^2; upper bound is the infinite GP — both $\Theta(n^2)$.)

Ex D. $T(n) = T(n-1) + T(n-2) + 1$ (Fibonacci-like) 📝 p. 6

```
Level i has at most 2^i nodes, each costing 1.
Shortest branch (n-2 each time) → depth n/2   → Ω(2^(n/2))
Longest  branch (n-1 each time) → depth n     → O(2^n)
So  Ω(2^(n/2)) ≤ T(n) ≤ O(2^n);  tight value is θ(φⁿ), φ = 1.618.
```

> ⚠️ **Trap:** An MCQ may offer "$\Theta(2^n)$" and "$O(2^n)$" — only the **O** is certainly right unless the question constructs it specially. Exact Fibonacci is $\Theta(\varphi^n)$.

Ex E. $T(n) = T(n/2) + T(n/3) + T(n/4) + 1$ 📝 p. 7

```
Level i has up to 3^i nodes, each cost 1.
Shortest branch (÷4): depth log₄ n → total ≥ 3^(log₄ n) = n^(log₄ 3) = n^0.79
Longest  branch (÷2): depth log₂ n → total ≤ 3^(log₂ n) = n^(log₂ 3) = n^1.58
So   Ω(n^0.79) ≤ T(n) ≤ O(n^1.58)
```

(Exact answer via Akra–Bazzi is ≈ n^1.08 — outside GATE syllabus, so GATE gives *bounds* here.) > 📝 *Your notes wrote the upper bound as n^1.5; the correct exponent is log₂3 ≈ 1.58.*

Ex F. General tree for `T(n) = aT(n/b) + c·nᵏ` 📝 p. 8

```
Level i: aⁱ sub-problems, each of size n/bⁱ, cost c(n/bⁱ)ᵏ  → level cost = c nᵏ (a/bᵏ)ⁱ
Number of levels = log_b n
Total = c nᵏ [1 + r + r² + … + r^(log_b n)]   where r = a / bᵏ
```

The behaviour of this GP is the whole Master theorem:

- $r < 1$ ($a < b^k$): decreasing GP → **root dominates** → $\Theta(n^k)$
- $r = 1$ ($a = b^k$): every level equal → $\Theta(n^k \log n)$

- r > 1 (a > b^k): increasing GP → **leaves dominate** → Θ(n^(log_b a))

GP shortcuts 📝 p. 6: $1 + c + c^2 + ... + c^n$ is **Θ(1)** if c < 1, **Θ(n)** if c = 1, **Θ(c^n)** if c > 1.

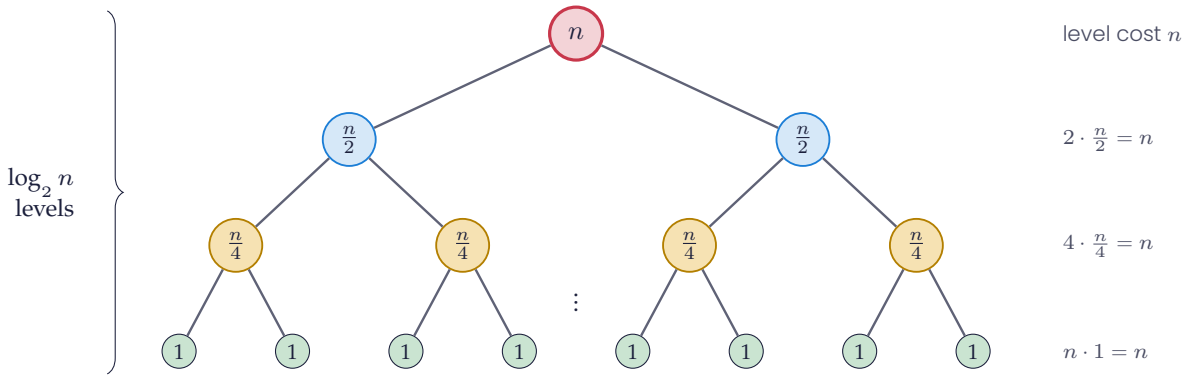


Figure 2.1. Recursion tree of $T(n) = 2T(n/2) + n$: every level costs n and there are $\log n$ levels, so the total is $\Theta(n \log n)$.

2.4 Method 3 — Master Theorem (the workhorse)

📝 Lecture notes pp. 8–10

For $T(n) = a \cdot T(n/b) + f(n)$ with **a ≥ 1, b > 1**. Compute the **watershed function** $n^{\log_b a}$ and compare it with f(n):

Case	Condition	Answer
1	f(n) = O(n^(log_b a − ε)) for some ε>0 (*f is polynomially smaller*)	**T(n) = Θ(n^(log_b a))**
2	f(n) = Θ(n^(log_b a)) (*same size*)	**T(n) = Θ(n^(log_b a) · log n)**
3	f(n) = Ω(n^(log_b a + ε)) for some ε>0 (*f is polynomially bigger*)	**T(n) = Θ(f(n))** (regularity condition a·f(n/b) ≤ c·f(n), c<1, holds for all normal functions)

> 🔑 **"Polynomially" is the whole catch.** One function must be bigger by a factor of **n^ε**, not just by a log factor. n^3 vs $n^2 \log n$ → n^3 is bigger by **n** → polynomially bigger ✅. n^2 vs $n^2 \log n$ → differ only by **log n** → **not polynomially comparable** ❌ → **basic Master fails** (use the extended version in 2.5).

Worked examples (all from your notes)

#	Recurrence	n^(log_b a)	Compare	Answer
1	$T(n) = 4T(n/2) + n^3$	n^2	$n^3 \gg n^2$ (Case 3)	**$\Theta(n^3)$**
2	$T(n) = 2T(n/2) + 2^n$	n	$2^n \gg n$ (Case 3)	**$\Theta(2^n)$**
3	$T(n) = 9T(n/3) + n^2 \log n$	n^2	n^2 vs $n^2 \log n$ — *not polynomially comparable*	basic Master ❌ → extended gives **$\Theta(n^2 \log^2 n)$**
4	$T(n) = 2T(n/3) + (\lg n)^2$	$n^{\log_3 2} \approx n^{0.63}$	$n^{0.63} \gg \log^2 n$ (Case 1)	**$\Theta(n^{\log_3 2})$**
5	$T(n) = 2T(n/2) + n$	n	equal (Case 2)	$\Theta(n \log n)$
6	$T(n) = 8T(n/2) + n^2$	n^3	$n^2 \ll n^3$ (Case 1)	$\Theta(n^3)$ (naïve matrix multiply)
7	$T(n) = 7T(n/2) + n^2$	$n^{(\log_2 7)} = n^{2.81}$	Case 1	$\Theta(n^{2.81})$ (Strassen)
8	$T(n) = T(n/2) + 1$	1	equal (Case 2)	$\Theta(\log n)$

> ⚡ **GATE Shortcut — the 10-second Master procedure**
>
> 1. Write down a, b, and f(n).
> 2. Compute $\log_b a$ (e.g. a=9,b=3 → 2; a=4,b=2 → 2; a=2,b=3 → 0.63).
> 3. If f is a clean power n^k: compare **k with log_b a**: k < → n^(log_b a); k = → $n^k \log n$; k > → n^k.
> 4. If f has a log attached ($n^k \log^p n$) go to the **extended** table immediately.
> 5. If f is exponential (2^n) or n! it's always Case 3.
> 6. **Master does NOT apply** when: a is not constant ($nT(n/2)$), a < 1, b isn't a *fraction* of n ($T(n-1)$), f(n) is negative, or when the pieces are unequal ($T(n/3) + T(2n/3)$).

2.5 Extended (Complete) Master Theorem

📝 Lecture notes p. 6, 10

For $T(n) = a \cdot T(n/b) + n^k \cdot \log^p n$, with a ≥ 1, b > 1, k ≥ 0, p any real number:

Condition	Sub-condition	Answer
$a > b^k$ (equivalently log_b a > k)	any p	**$\Theta(n^{\log_b a})$**
$a = b^k$ (log_b a = k)	**$p > -1$**	**$\Theta(n^k \cdot \log^{p+1} n)$**
	$p = -1$	**$\Theta(n^k \cdot \log \log n)$**
	$p < -1$	**$\Theta(n^k)$**
$a < b^k$ (log_b a < k)	**$p \geq 0$**	**$\Theta(n^k \log^p n)$**
	$p < 0$	**$\Theta(n^k)$**

(Your notes write $O(n^k)$ for the last row; the tight answer is $\Theta(n^k)$ because f itself is a lower bound.)

> ⚡ **Memory hook:** *Bottom heavy ($a > b^k$) → leaves win. Balanced ($a = b^k$) → add one to the log power (and $p = -1$ is the special log-log case). Top heavy ($a < b^k$) → root wins.*

Worked examples 📝 pp. 5–7

Recurrence	a, b, k, p	Case	Answer
$9T(n/3) + n^2 \log n$	9, 3, 2, 1	a = b^k (9=3^2), p=1 > −1	**Θ(n² log² n)**
$2T(n/2) + n/\log n$	2, 2, 1, −1	a = b^k, p = −1	**Θ(n log log n)**
$4T(n/2) + n/\log^2 n$	4, 2, 1, −2	a = 4 > b^k = 2	**Θ(n²)**
$2T(n/2) + n \log n$	2, 2, 1, 1	a = b^k, p=1	**Θ(n log² n)**
$8T(n/2) + n^3 \log n$	8, 2, 3, 1	a = b^k (8=2^3), p=1	**Θ(n³ log² n)**
$2T(n/4) + \sqrt{n}$	2, 4, ½, 0	a = b^k (2 = 4^½), p=0	**Θ(√n log n)**
$3T(n/2) + n^2/\log n$	3, 2, 2, −1	a=3 < b^k=4, p<0	**Θ(n²)**

Why $T(n) = 2T(n/2) + n/\log n$ gives n log log n (proof from your notes, p. 5–6):

```
Unrolling: T(n) = 2^k T(n/2^k) + n[ 1/log n + 1/log(n/2) + … + 1/log(n/2^(k-1)) ]
With k = log n the bracket is  1/k + 1/(k-1) + … + 1/1 = H_k  (harmonic series ≈ ln k)
So T(n) ≈ n · ln(log n) = θ(n log log n)
```

2.6 Method 4 — Change of variable (for √n and similar)

📝 Lecture notes pp. 7–9

When the argument shrinks by a *root* ($\sqrt{n}$, $n^{1/3}$), substitute **n = 2^m**, so m = log n, and √n = 2^(m/2). Define **S(m) = T(2^m)**. The recurrence becomes an ordinary "divide-by-b" recurrence in m. Solve with Master. Finally convert back: m = log n.

Ex 1. $T(n) = T(\sqrt{n}) + \log n$, T(2) = 1 $S(m) = S(m/2) + m$ → a=1,b=2,k=1, a<b^k → **Θ(m) = Θ(log n)**. (Direct unrolling in the notes: 1 + 2 log n − 1 → Θ(log n).)

Ex 2. $T(n) = T(\sqrt{n}) + \sqrt{n}$ $S(m) = S(m/2) + 2^{m/2}$ → f is exponential, dominates → S(m) = Θ(2^(m/2)) = **Θ(√n)**.

Ex 3. $T(n) = T(\sqrt{n}) + \log\log n$ $S(m) = S(m/2) + \log m$ → this is Σ log(m/2^i) over log m terms ≈ ½ $\log^2$ m → **Θ((log log n)²)**.

Ex 4. $T(n) = T(\sqrt{n}) + n^2$ $S(m) = S(m/2) + 2^{2m}$ → f = 2^(2m) dominates (Case 3) → Θ(2^(2m)) = **Θ(n²)**.

Ex 5. $T(n) = 12 \cdot T(n^{1/3}) + (\log n)^2$ $S(m) = 12 \cdot S(m/3) + m^2$ → m^(log₃12) = m^2.26 > m² (Case 1) → Θ(m^2.26) = **Θ((log n)^(log₃ 12))**.

Ex 6 (classic). $T(n) = 2T(\sqrt{n}) + \log n$ $S(m) = 2S(m/2) + m$ → **Θ(m log m) = Θ(log n · log log n)**.

> 📘 ⚡ **GATE Shortcut for T(√n) family:** after substitution the answer is always some function of **log n** (e.g. log n, log log n, (log n)^c). Only when f(n) is polynomial/exponential in n

does the answer come back as a polynomial in n.

2.7 Master flow-chart (print this)

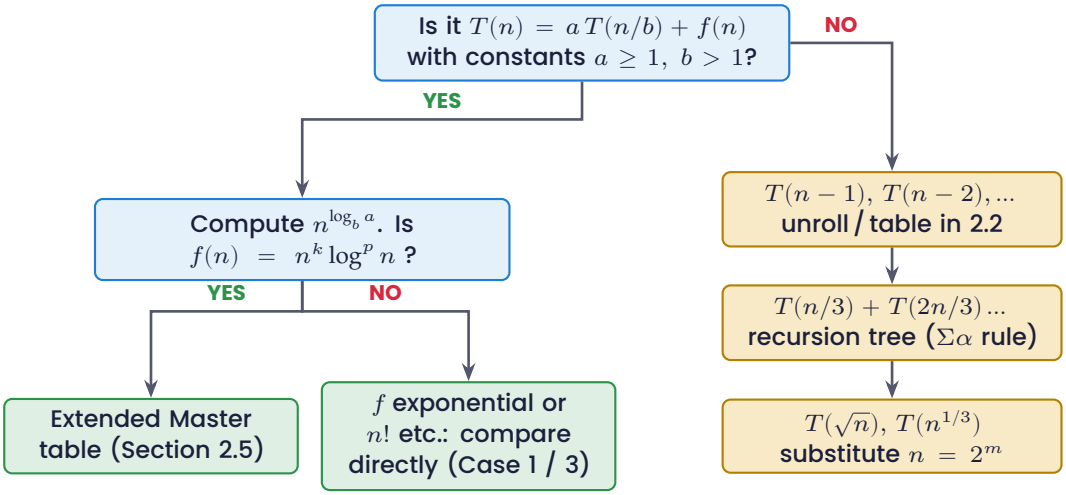


Figure 2.2. Master flow-chart: pick the right tool for the recurrence at hand.

2.8 Practice — Chapter 2 (do all before moving on)

🎯 TRY IT YOURSELF -- cover the answers first

#	Recurrence	Answer	Key step
1	$T(n) = 3T(n/2) + n^2$	**$\Theta(n^2)$**	$\log_2 3 = 1.58 < 2$
2	$T(n) = 4T(n/2) + n^2$	**$\Theta(n^2 \log n)$**	$a = b^k$, $p=0$
3	$T(n) = 16T(n/4) + n$	**$\Theta(n^2)$**	$\log_4 16 = 2 > 1$
4	$T(n) = T(n/2) + n$	**$\Theta(n)$**	$a < b^k$
5	$T(n) = 2T(n/2) + n/\log n$	**$\Theta(n \log\log n)$**	$p = -1$
6	$T(n) = T(n-1) + \log n$	**$\Theta(n \log n)$**	$\Sigma \log i = \log n!$
7	$T(n) = 2T(n-1) + 1$	**$\Theta(2^n)$** (exactly $2^n - 1$ for $T(0)=0$)	Hanoi
8	$T(n) = T(n/2) + T(n/4) + T(n/8) + n$	**$\Theta(n)$**	$\Sigma\alpha = 7/8 < 1$
9	$T(n) = T(n/4) + T(n/2) + n$	**$\Theta(n)$**	$\Sigma\alpha = 3/4 < 1$, root dominates
10	$T(n) = 2T(\sqrt{n}) + \log n$	**$\Theta(\log n \cdot \log\log n)$**	$n = 2^m$
11	$T(n) = T(n/3) + T(2n/3) + n$	**$\Theta(n \log n)$**	$\Sigma\alpha = 1$
12	$T(n) = 7T(n/2) + n^2$	**$\Theta(n^{\log_2 7})$**	Strassen
13	$T(n) = 5T(n/2) + n^2 \log n$	**$\Theta(n^{\log_2 5})$**	$a = 5 > 4$
14	$T(n) = T(n-1) + 1/n$	**$\Theta(\log n)$**	harmonic
15	$T(n) = nT(n-1)$, $T(0)=1$	**$n! = \Theta(n!)$**	product

Common 1-mark traps

1. $T(n) = 2T(n/2) + n^2$ is **$\Theta(n^2)$**, *not* $n^2 \log n$ (f dominates).
2. $T(n) = T(n-1) + n$ is $\Theta(n^2)$, never $\Theta(n \log n)$.
3. Using Master on $T(n) = T(n-1) + \ldots$ is *illegal*.
4. $T(n) = 2T(n/2) + n \log n$ is $\Theta(n \log^2 n)$ — not $\Theta(n \log n)$ and not $\Theta(n^2)$.

PART

II

Divide & Conquer and Sorting

IN THIS PART

Ch 3 Divide & Conquer, Merge Sort, Quick Sort and the Sorting Family

CHAPTER 3

Divide & Conquer, Merge Sort, Quick Sort and the Sorting Family

📝 Notes pp. 17–42 (D&C) and pp. 115–119 (sorting). Every example in those pages is below.

3.1 The D&C idea

Many problems are solved by one 3-step pattern:

1. **Divide** — split the problem into smaller sub-problems of the same type.
2. **Conquer** — solve each sub-problem *recursively* (stop at a tiny base case).
3. **Combine** — merge the sub-answers into the final answer.

The running time then *always* looks like `T(n) = a·T(n/b) + (divide+combine cost)` → use Chapter 2.

3.2 Maximum of an array (the ``hello world'' of D&C)

📝 Lecture notes p. 18

PSEUDOCODE

```
Maximum(A, l, h):
    if l == h: return A[l]
    m  = l + (h - l)/2
    m1 = Maximum(A, l, m)
    m2 = Maximum(A, m+1, h)
    return max(m1, m2)
```

$T(n) = 2T(n/2) + 1$ → Master (a=2 > b^k=1) → **Θ(n)**. Same as a simple loop (n−1 comparisons) — D&C isn't always faster!

3.3 Merge Sort

📝 Lecture notes pp. 18–21

Idea: split array in two halves → sort each recursively → *merge* two sorted halves.

PSEUDOCODE

```
mergesort(A, l, h):
    if l < h:                       // (notes write the base-case test as l == h → return)
```

```
        m = (l + h) / 2
        mergesort(A, l, m)
        mergesort(A, m+1, h)
        merge(A, l, m, h)
```

The merge step (the real work): copy the halves into LA and RA; keep two pointers; repeatedly copy the smaller front element; when one half runs out copy the rest of the other.

PSEUDOCODE

```
merge(A,l,m,h):
    LA = A[l..m];  RA = A[m+1..h];  i=j=k=0
    while i<|LA| and j<|RA|:
        if LA[i] < RA[j]: A[k++] = LA[i++]
        else:             A[k++] = RA[j++]
    copy leftovers of LA, then of RA
```

- Merging two sorted lists of sizes p and q costs **at most p+q−1 comparisons** (worst) and Θ(p+q) time.
- **Recurrence:** $T(n) = 2T(n/2) + n \rightarrow$ **Θ(n log n)** in best, average **and** worst case.
- Needs **Θ(n) extra space** (not in-place). It is **stable**.

Why Θ(n log n) (tree picture): log n levels, each level does ~n work in total → n log n. 📝 p. 20 shows the unrolled proof: $T(n) = 2^k T(n/2^k) + k \cdot n$ with k = log n.

Iterative (bottom-up) merge sort 📝 p. 23: start with blocks of size 1, merge neighbouring pairs → blocks of 2 → 4 → 8 ... until one block. $\log n$ rounds × $O(n)$ per round → **Θ(n log n)**, and no recursion stack.

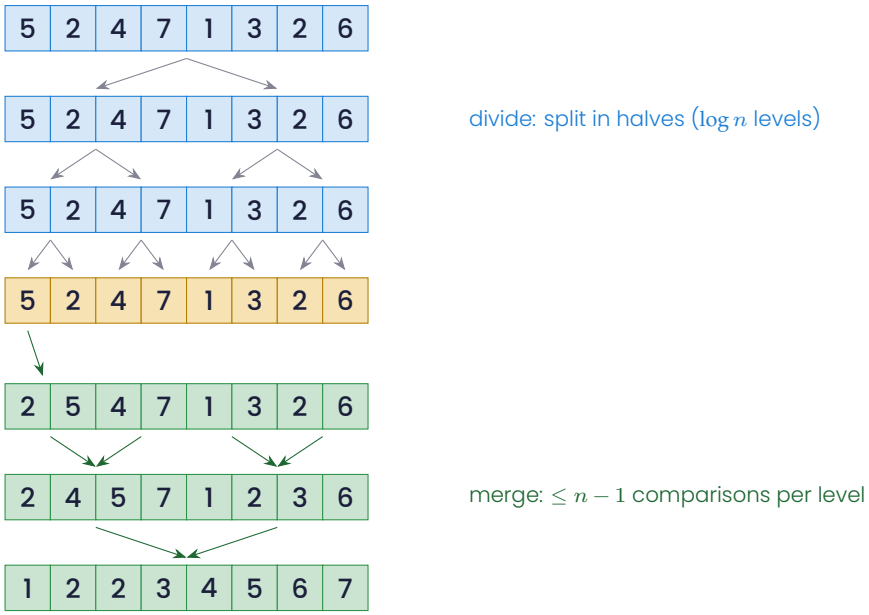


Figure 3.1. Merge sort on 8 elements: dividing costs nothing; every merge level does $\Theta(n)$ work in total, giving $\Theta(n \log n)$.

3.4 Merge-sort variants asked in your notes

📝 Lecture notes pp. 20–23

These are classic GATE twists — learn the *reasoning*.

Variant	Recurrence	Answer	Reasoning
Split into **3** parts, merge three sorted arrays of size n/3: how many comparisons (worst) in the merge?	merge two first, then the result with the third	**(2n/3 − 1) + (n − 1) = 5n/3 − 2**	first merge n/3+n/3 → 2n/3−1; second merge 2n/3+n/3 → n−1
Recurrence for 3-way merge sort	$T(n) = 3T(n/3) + \Theta(n)$	**Θ(n log n)**	$a=b^k$
Merge takes only **Θ(1)** (hypothetical)	$T(n) = 2T(n/2) + \Theta(1)$	**Θ(n)**	$a > b^k$
Divide so left part = 1 element, right = n−1	$T(n) = T(n-1) + \Theta(n)$	**Θ(n²)**	unrolled sum
Divide into **K equal parts**, K constant	$T(n) = K \cdot T(n/K) + \Theta(n)$ (merging K arrays of size n/K costs K(n−1)/... = Θ(n) for constant K)	**Θ(n log n)**	$a=K=b^k$
Divide at a **random point**, worst case	worst split is 1 : n−1	**Θ(n²)**	$T(n) = T(1) + T(n-1) + \Theta(n)$

> 📝 *Your notes wrote the 3-way merge comparison total as "n−2"; recomputing gives 5n/3 − 2.*

Efficient algorithm questions (from notes p. 22–23):

- **Sort a bitonic array** (increasing up to index t, then decreasing): walk from the left until the next value drops → finds t in O(t). The two pieces are sorted (reverse the second) → merge → **Θ(n)** total.
- **"K-even-mixed" array** (exactly K even integers; the odd integers already appear in sorted order), K = n/log n: separate evens/odds in Θ(n); sort the K evens: $K \log K = (n/\log n) \cdot \log n = \Theta(n)$; merge the two sorted lists Θ(n). **Total Θ(n).** > ⚡Pattern: *"part is already sorted + small part is unsorted"* → sort the small part in $k \log k$, then merge.

3.5 Merging K sorted arrays (total n elements, each n/K long)

📝 Lecture notes pp. 24–26

Four methods — memorise the table:

Method	How	Time
1. **Repeated merge**	merge $A_1+A_2 \to$ then $+A_3 \to \dots$	$n/K\cdot(2+3+\dots+K) = \mathbf{\Theta(nK)}$
2. **Scan for min**	each step look at the K front elements	$(K-1)$ comparisons × $(n-1)$ steps = $\mathbf{\Theta(nK)}$
3. **Min-heap of K fronts**	build heap $\Theta(K)$; each of n steps: pop $\Theta(1)$+push $\Theta(\log K)$	$\mathbf{\Theta(n \log K)}$
4. **Pairwise (tournament) merge**	merge in pairs → K/2 arrays → … → 1; log K rounds each $\Theta(n)$	$\mathbf{\Theta(n \log K)}$

> ⚡Methods 3 and 4 are optimal: $\mathbf{\Theta(n \log K)}$. For $K = 2$ this reduces to $\Theta(n)$; for $K = n$ it equals $\Theta(n \log n)$ (it *is* merge sort).

3.6 Stable and in-place sorting

📝 Lecture notes p. 27

- **Stable:** equal keys keep their **original relative order** in the output. Example: `3a 1 2 3b 4 3c` → `1 2 3a 3b 3c 4` (the three 3's stay in order a, b, c).
- **In-place:** uses only $\mathbf{\Theta(1)}$ extra space (apart from the input).

Algorithm	Best	Average	Worst	Stable?	In-place?
Bubble	$\Theta(n)$ (with early exit)	$\Theta(n^2)$	$\Theta(n^2)$	✅	✅
Insertion	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	✅	✅
Selection	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	❌	✅
Merge	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	✅	❌ ($\Theta(n)$ space)
Quick	$\Theta(n \log n)$	$\Theta(n \log n)$	$\mathbf{\Theta(n^2)}$	❌	✅ ($O(\log n)$ stack)
Heap (Ch 10)	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	❌	✅
Counting / Radix (non-comparison)	$\Theta(n+k)$ / $\Theta(d(n+k))$	same	same	✅	❌

3.7 Maximum, minimum and second maximum — counting comparisons

📝 Lecture notes pp. 28–31

GATE loves "how many comparisons".

Finding the maximum

- Naive loop: **n−1** comparisons.
- Tournament (knockout) method: $n/2 + n/4 + \ldots + 1 = \mathbf{n-1}$ — same count, but the tree structure helps in the next problem.

Finding max and second max

- Naive (compare each element with both max and smax): worst $\mathbf{(n-2)\cdot 2 + 1 = 2n-3}$.
- **Tournament method:** the second-largest must have lost **directly to the largest** (everyone else was beaten by someone who isn't the champion). The champion played exactly **log n** matches ⇒ the second max is among its log n "victims".
  - Step 1: find max by knockout: n−1.
  - Step 2: max among log n victims: log n − 1.
  - **Total = n + ⌈log n⌉ − 2.** 📝 p. 29 example `[10,15,8,2,1,3,6,19]`: round 1 winners 15, 8, 3, 19 → round 2: 15, 19 → final 19. Champion 19 beat **6, 3 and 15** (log 8 = 3 victims) → the largest of those, **15**, is the second maximum. Comparisons = 7 + 2 = 9 = n + log n − 2 ✓.

Finding max and min together

- Naive: $\mathbf{2(n-1) = 2n-2}$.
- **Pair method:** compare elements in pairs (n/2 comparisons); put the winner of each pair in a "max array", the loser in a "min array"; then find max of the max-array (n/2−1) and min of the min-array (n/2−1). Total = $n/2 + 2(n/2 - 1) = \mathbf{3n/2 - 2}$ ($\lceil 3n/2 \rceil - 2$ in general). 📝 p. 30 example: `[10,5,21,23,48,32,21,6]` → max-array `[10,23,48,21]` → max **48**; min-array `[5,21,32,6]` → min **5**. Total comparisons = 4 (pairs) + 3 + 3 = 10 = 3n/2 − 2 ✓.

> ⚡ **Memory trio:** max → n−1 · max+min → 3n/2−2 · max+2nd max → n+log n−2.

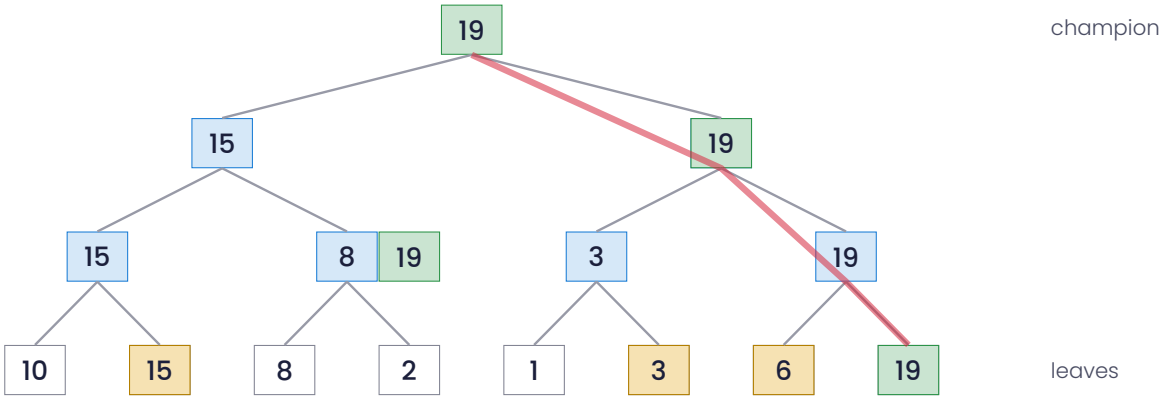


Figure 3.2. *Knockout tournament on* $[10, 15, 8, 2, 1, 3, 6, 19]$*:* $n - 1 = 7$ *comparisons find the maximum, then* $\log n - 1 = 2$ *more find the second maximum (total* $n + \log n - 2 = 9$*).*

3.8 Counting inversions

📝 Lecture notes pp. 32–34

Definition: indices i < j with **A[i] > A[j]** form an inversion. (An inversion = "a pair out of order"; sorted array has 0.)

- **Maximum possible inversions** in an array of n: reverse-sorted → **n(n−1)/2**. For n = 6: 5+4+3+2+1 = **15**.
- **Lemma (notes p. 32):** If (i, j) is an inversion then the array has **at least j − i inversions**. *Proof idea:* every position k between i and j either is smaller than A[i]... or larger than A[j]... so k forms an inversion with i or with j (three cases on the value of A[k]).
- **Link to sorting:** insertion sort performs **exactly #inversions swaps**; bubble sort also.

Method 1 (brute force): two nested loops, count A[i] > A[j] → $\Theta(n^2)$ (n(n−1)/2 comparisons).

Method 2 (merge sort, Θ(n log n)): `inversions = left_inv + right_inv + cross_inv`. While merging two sorted halves, every time an element of the **right** half is placed *before* the remaining elements of the left half, it forms an inversion with **each** of those remaining left elements.

Worked example 📝 p. 33: `A = [5, 7, 9, 3, 1, 2]`

```
Split → [5 7 9] [3 1 2]
 [5 7 9]: sorted already → 0 inversions
 [3 1 2] → [3] [1 2]: merge 3 with [1 2]: 1 jumps over 1 element (3) , 2 jumps over 1 → 2
  ↪  inversions ; [1 2] sorted
 Merge [5 7 9] with [1 2 3]:
     1 placed with 3 left-elements remaining → +3
     2 placed with 3 left-elements remaining → +3
     3 placed with 3 left-elements remaining → +3
 Cross = 9
Total = 0 + 2 + 9 = 11
```

(Check by listing: (5,3)(5,1)(5,2)(7,3)(7,1)(7,2)(9,3)(9,1)(9,2)(3,1)(3,2) = **11** ✓.) Recurrence stays $T(n) = 2T(n/2) + \Theta(n)$ → **Θ(n log n)**.

3.9 Closest pair of points in 2-D

📝 Lecture notes pp. 34–35

Problem: given n points in the plane, find the pair with the **minimum distance**.

- Brute force: all pairs → $\Theta(n^2)$.
- **D&C:** sort points by x (and also keep them sorted by y). Split by a vertical line into two halves; recursively get δ_L and δ_R; let $\delta = \min(\delta_L, \delta_R)$. The only unseen pairs straddle the line — and they must lie within a vertical **strip of width** δ on each side. For a point in the left strip you only need to check points of the right strip within a **δ-box**, and a box of that size can only contain a **constant number** ($\leq$ ~6–7) of points. So the combine step is $\Theta(n)$.
- $T(n) = 2T(n/2) + \Theta(n)$ plus an initial $\Theta(n \log n)$ sort → **$\Theta(n \log n)$**.

3.10 Fast exponentiation a^b

📝 Lecture notes pp. 35–36

- Naive: multiply a, b times → **$\Theta(b)$**.
- **D&C:** $a^b = a^{b/2} \cdot a^{b/2}$ (times one more a if b is odd). *Compute a^(b/2) only once* and reuse it.

PSEUDOCODE

```
exponent(a,b):
    if b == 0: return 1
    if b == 1: return a
    h = exponent(a, b/2)
    return b even ? h·h : h·h·a
```

$T(b) = T(b/2) + 1$ → **$\Theta(\log b)$** multiplications. > ⚠️ **Trap:** If you write `exponent(a,b/2) * exponent(a,b/2)` you make **two** calls and the recurrence becomes $T(b) = 2T(b/2) + 1 = \Theta(b)$ — no improvement!

3.11 Matrix multiplication

📝 Lecture notes pp. 36–37

- Definition method: three loops → **$\Theta(n^3)$**.
- **D&C:** cut each n×n matrix into four (n/2)×(n/2) blocks. The product needs **8** block-multiplications plus additions of cost $(n/2)^2$ each ×4 = $\Theta(n^2)$: $T(n) = 8T(n/2) + n^2$ → a=8 > b^k=4 → **$\Theta(n^3)$** (no gain).
- **Strassen (🆕 Added):** cleverly uses **7** multiplications → $T(n) = 7T(n/2) + n^2$ → **$\Theta(n^{\log_2 7})$ = $\Theta(n^{2.81})$**.

3.12 Quick Sort

📝 Lecture notes pp. 37–40

Idea: pick a **pivot**; rearrange so that everything smaller is to its left, everything larger to its right, pivot in its final place (the *partition* step); then recurse on both sides. **Almost all work is done in the partition (the "divide"), the combine step is free.**

PSEUDOCODE

```
QS(A, l, h):
    if l < h:
        p = Partition(A, l, h)
        QS(A, l, p-1);  QS(A, p+1, h)

Partition(A, l, h):// Lomuto scheme (pivot = last element)
    pvt = A[h];  i = l - 1
    for j = l .. h-1:
        if A[j] ≤ pvt:  i++;  swap(A[i], A[j])
    swap(A[i+1], A[h]);  return i+1
```

Partition scans the array once → **Θ(n)** comparisons (exactly n−1 for a size-n array).

Cases

Case	When	Recurrence	Time
Best	pivot always the median	$T(n) = 2T(n/2) + \Theta(n)$	**Θ(n log n)**
Worst	pivot always the min or max (e.g. already sorted / reverse sorted input with first/last pivot)	$T(n) = T(n-1) + T(1) + \Theta(n)$	**Θ(n²)**
Average	random pivot		**Θ(n log n)**
Any **constant-ratio split** (even 1 : 99)	$T(n) = T(n/100) + T(99n/100) + n$	tree: every level costs n; depth between log₁₀₀ n and log_{100/99} n → **Θ(n log n)**	

- Quick sort is **in-place** (O(log n) stack on average) and **not stable**.
- If you always pick the **true median** as pivot, even the worst case is Θ(n log n) (but finding the median costs Θ(n) with the median-of-medians algorithm, so total is still Θ(n log n)).

Worked partition trace (to practise NAT questions) — `A = [2, 8, 7, 1, 3, 5, 6, 4]`, pivot = 4:

```
j=0: 2 ≤ 4 → i=0, swap(A0,A0)          [2 8 7 1 3 5 6 4]
j=1: 8 > 4
j=2: 7 > 4
j=3: 1 ≤ 4 → i=1, swap(A1,A3)          [2 1 7 8 3 5 6 4]
j=4: 3 ≤ 4 → i=2, swap(A2,A4)          [2 1 3 8 7 5 6 4]
j=5: 5, j=6: 6  → no change
final: swap(A3, pivot)                 [2 1 3 4 7 5 6 8]   pivot index = 3
Swaps = 4 (3 in the loop + 1 final).  Comparisons = 7 (= n-1).
```

Probability question 📝 p. 41: *"What is the probability that Partition splits an array so that the smaller part has at least $n/5$ elements?"* — In sorted order the pivot must have rank between n/5 and 4n/5, i.e. **n − 2n/5 = 3n/5 good choices out of n** → probability **1 − 2/5 = 3/5**.

3.13 The Selection problem (k-th smallest)

📝 Lecture notes pp. 41–42

Quick-select: pick a pivot, partition, look at the pivot's final index p:

- p = k → done (return A[p]).
- p > k → k-th smallest lies in the **left** part → recurse left.
- p < k → recurse **right** only (with k adjusted). You recurse into **one** side only (unlike quick sort).

Case	Time
Best (first pivot is the answer)	one partition = $\Theta(n)$ (your notes write $\Theta(1)$, counting only the recursion)
Average	$T(n) = T(n/2) + \Theta(n)$ → **$\Theta(n)$**
Worst	$T(n) = T(n-1) + \Theta(n)$ → **$\Theta(n^2)$**

📝 p. 42 example: `A = [1,7,5,2,8,3,4,6]`, **k = 4 (0-indexed)** → answer **5** (sorted: 1 2 3 4 5 6 7 8; index 4 = 5). Trace: pivot 1 lands at index 0 (< k, go right) → pivot 6 lands at index 5 (> k, go left) → pivot 5 at index 4 = k ✓.

Median-of-medians (🆕 clarified): group the elements into chunks of 5, take each chunk's median, take the median of those medians as pivot. This guarantees a split no worse than ≈ 30:70, so quick-select becomes **worst-case $\Theta(n)$** and quick-sort with this pivot is **worst-case $\Theta(n \log n)$**. (With groups of 3 the guarantee fails and you only get $\Theta(n \log n)$.)

3.14 Binary search

📝 Lecture notes p. 43

PSEUDOCODE

```
BS(A, l, h, x):
    if l ≤ h:
        m = (l+h)/2
        if A[m] == x: return m
        else if x < A[m]: return BS(A, l, m-1, x)
        else:             return BS(A, m+1, h, x)
    return -1
```

$T(n) = T(n/2) + 1$ → **$\Theta(\log n)$**; worst case ≈ $\lfloor \log_2 n \rfloor + 1$ comparisons. Requires a **sorted** array (or a monotone structure). > ⚡ GATE 2021 S2 asked "worst-case arithmetic operations of recursive binary search on n elements" — answer **$\Theta(\log n)$** (constant work per level).

3.15 The sorting algorithms in detail

📝 Lecture notes pp. 115–119

Bubble sort — repeatedly compare neighbours, swap if out of order; after pass i the i largest are in place.

- *Best:* already sorted → 0 swaps, **n−1** comparisons (if we stop when a pass makes no swap) → $\Theta(n)$.
- *Worst:* reverse sorted → comparisons = swaps = $\mathbf{n(n-1)/2} \to \Theta(n^2)$. In-place, stable.

Insertion sort — keep A[0..i] sorted; insert A[i+1] into its place.

- *Best:* sorted → 0 swaps, n−1 comparisons → $\Theta(n)$.
- *Worst:* reverse sorted → swaps = comparisons = $0+1+\ldots+(n-1) = \mathbf{n(n-1)/2}$. **#swaps = #inversions** always.
- In-place, stable. Best choice for **small or nearly-sorted** input.

Selection sort — find the largest of n, put it last; repeat for n−1, ...

- Always $\mathbf{n(n-1)/2}$ **comparisons**, in *all* cases → $\Theta(n^2)$.
- **Swaps ≤ n−1** (best 0, worst n−1) — the fewest swaps of the quadratic sorts.
- In-place, **unstable** (long-distance swap breaks equal-key order: `[2a, 2b, 1]` → swap 2a↔1 → `[1, 2b, 2a]`).

Theorem — comparison-sort lower bound 📝 p. 119: *Any comparison-based sorting algorithm needs* $\boldsymbol{\Omega(n \log n)}$ *comparisons in the worst case. Why:* the algorithm's behaviour is a binary decision tree whose leaves are the n! possible orderings; a binary tree with n! leaves has height $\geq \log_2(n!) = \Theta(n \log n)$. (For n = 4: $\lceil \log_2 24 \rceil = \mathbf{5}$ comparisons minimum.) > ⚡ Therefore merge sort and heap sort are **asymptotically optimal**; and any claim "sort in O(n) using comparisons" is false. Counting/radix/bucket sort beat it only because they are *not* comparison-based (they use the key's value as an index).

Which sort should I choose? (🆕 PYQ "sorting theory" pattern)

Situation	Best choice	Why
Nearly sorted / small n	Insertion	$\Theta(n + \#\text{inversions})$
Must be stable & guaranteed $\Theta(n \log n)$	Merge	stable, no bad case
Memory-tight, guaranteed $\Theta(n \log n)$	Heap	in-place
Fastest on average in RAM	Quick (randomised pivot)	tiny constant
Keys are integers in range 0..k, k = O(n)	Counting / Radix	$\Theta(n)$
Each element is at most k positions from its place	Insertion $\Theta(nk)$ or **min-heap of k+1 elements $\Theta(n \log k)$**	
Minimise number of **writes/swaps**	Selection	$\leq n-1$ swaps
Sort linked list	Merge	no random access needed

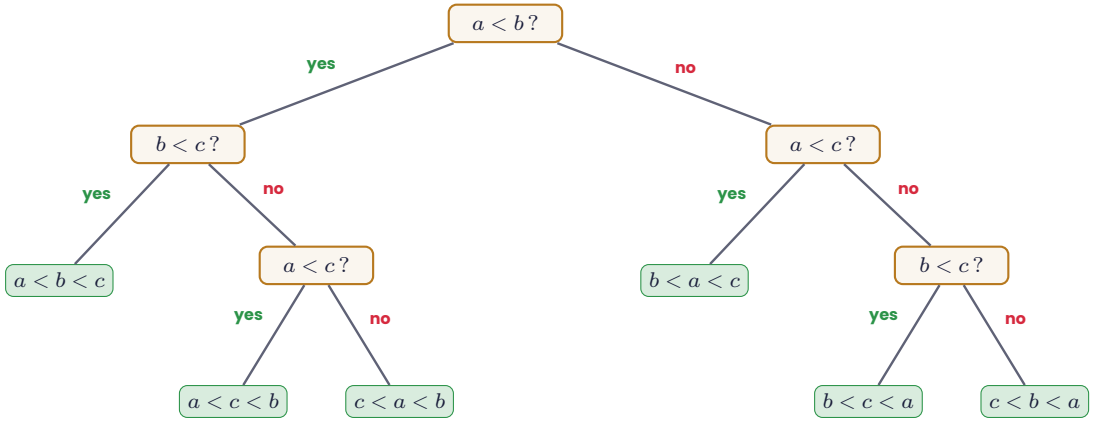


Figure 3.3. Decision tree of a comparison sort on 3 elements: $3! = 6$ leaves force a height of at least $\lceil \log_2 6 \rceil = 3$ comparisons; in general $\geq \log_2(n!) = \Omega(n \log n)$.

3.16 Practice — Chapter 3

🎯 TRY IT YOURSELF -- cover the answers first

1. **Max # of inversions in a 10-element array?** → $10 \cdot 9/2 = \mathbf{45}$.
2. **Insertion-sort swaps on `[4, 3, 2, 1]`?** → inversions = 6 → **6**.
3. **Inversions of `[3, 1, 2, 5, 4]`?** → (3,1)(3,2)(5,4) = **3**.
4. **Quick sort on an already sorted array with last-element pivot → $\Theta(n^2)$**; number of comparisons = (n−1)+(n−2)+…+1 = n(n−1)/2.
5. **Comparisons to find max & min of 100 elements (pair method)?** → $3 \cdot 100/2 - 2 = \mathbf{148}$.
6. **Comparisons to find the second smallest of 16 elements (tournament)?** → $16 + \log_2 16 - 2 =$ **18**.
7. **Merge two sorted lists of sizes 7 and 5, worst-case comparisons?** → 7+5−1 = **11**.
8. **K = 8 sorted arrays, total n elements; best method?** → heap or pairwise: **$\Theta(n \log 8) = \Theta(3n)$**.
9. **Recurrence for quick sort with split 1:3?** → $T(n) = T(n/4) + T(3n/4) + n$ → $\Theta(n \log n)$.
10. **True/False:** (a) Quick sort is stable ✗ (b) Merge sort is in-place ✗ (c) Insertion sort best case is $\Theta(n)$ ✓ (d) Heap sort is stable ✗ (e) Selection sort does $\Theta(n^2)$ comparisons even on sorted input ✓.

PART

III

Graphs

🟠 the highest-scoring cluster: 2.06 markspaper

IN THIS PART

Ch 4 Graph Basics, DFS and BFS

Ch 5 Shortest Paths: Greedy Thinking, Dijkstra, Bellman-Ford, Floyd-Warshall

Ch 6 Minimum Spanning Trees

CHAPTER 4

Graph Basics, DFS and BFS

📝 Notes pp. 44–60. GATE asked BFS/DFS statements in 2018, 2023, 2024 (×2), 2025 (×2), 2026 — almost every year.

4.1 Graph vocabulary from scratch

A **graph** G = (V, E) is a set of **vertices** (dots) V and **edges** (links) E.

- **Undirected** edge {u,v}: a two-way road. **Directed** edge u→v: a one-way road.
- **Weighted** graph: every edge has a number (distance, cost).
- **Path:** sequence of vertices joined by edges. **Cycle:** a path that returns to its start. **Simple** path: no repeated vertex.
- **Connected** (undirected): a path exists between every pair. **Strongly connected** (directed): a directed path from every vertex to every other.
- **Tree:** connected + no cycle ⇒ exactly **n−1 edges**. **DAG:** Directed Acyclic Graph.
- **Degree** of a vertex = number of edges at it (directed: in-degree & out-degree). **Σ degrees = 2E** (handshake lemma).
- **Dense** graph: $E \approx V^2$. **Sparse** graph: $E \approx V$.

Handy counts (memorise):

Fact	Value
Max edges, simple undirected graph	$n(n-1)/2$
Max edges, simple directed graph	$n(n-1)$
Max edges in a DAG	$n(n-1)/2$
Min edges for a connected graph	$n-1$
Spanning trees of the complete graph K_n (Cayley)	n^{n-2}
Number of cuts (2 non-empty parts) of n vertices	$2^{n-1}-1$

4.2 How a graph is stored

Lecture notes p. 44

Adjacency Matrix (AM): a V×V table; entry [u][v] = 1 if edge u–v exists. **Adjacency List (AL):** for each vertex, a list of its neighbours. (Notes example: square a–b–c–d with diagonal b–d. AM rows: a=0101, b=1011, c=0101, d=1110.)

	Adjacency Matrix	Adjacency List
Space	$\Theta(V^2)$	$\Theta(V+E)$
Is edge u→v present?	**$\Theta(1)$**	$\Theta(\deg u)$
List all neighbours of v	$\Theta(V)$	**$\Theta(\deg v)$**
List all edges	$\Theta(V^2)$	$\Theta(V+E)$
Insert edge	$\Theta(1)$	$\Theta(\deg u)$ ($\Theta(1)$ at the head of the list if duplicates are allowed)
Delete edge	$\Theta(1)$	$\Theta(\deg u)$

⚡ **Dense vs sparse comparison trick** (📝 p. 44): to compare complexities such as O(V^(3/2)) and O(E): if **dense** (E ≈ V²) then V^(3/2) < E; if **sparse** (E ≈ V) then V^(3/2) > E. Always substitute E ≈ V² or E ≈ V.

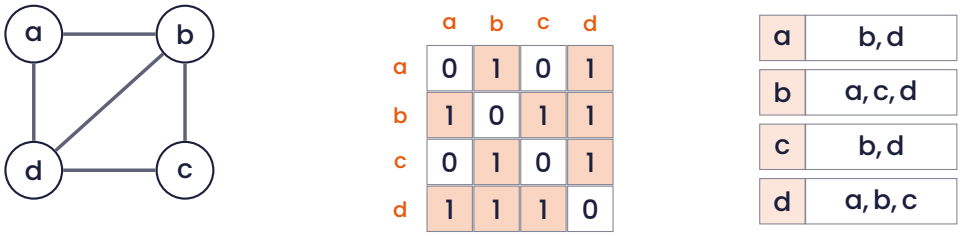


Figure 4.1. The same graph stored as an adjacency matrix ($\Theta(V^2)$ space) and as adjacency lists ($\Theta(V+E)$ space).

4.3 Depth-First Search (DFS)

Lecture notes pp. 45–48

Idea: go as deep as possible along one path; when stuck, **backtrack** and try the next unexplored edge. (Like exploring a maze with a ball of thread.) It uses a **stack** (explicit, or the recursion stack).

Timestamps (the secret weapon behind every DFS application):

- **Start (discovery) time `d[v]`**: the moment v is first visited.
- **Finish time `f[v]`**: the moment we are completely done with v and leave it for the last time. Use a single counter that ticks at every start *and* every finish.

📝 p. 45 example (source a; tree a→b, b→c, b→d, d→e, d→f): $a0/11, b1/10, c2/3, d4/9, e5/6, f7/8$.

Implementations

PSEUDOCODE

```
DFS(G, src, seen):// recursive
    process(src); seen[src] = true
    for each edge (src, v):
        if !seen[v]: DFS(G, v, seen)

DFS(G, src):// iterative
    stack = [src]; seen[src] = true
    while stack not empty:
        u = pop(stack); process(u)
        for v in Adj[u]: if !seen[v]: seen[v]=true; push(v)
```

Complexity: each vertex is processed once and each edge examined once (twice if undirected): $\Sigma(1 + deg(v)) = V + E \rightarrow \mathbf{\Theta(V + E)}$ with adjacency lists, $\mathbf{\Theta(V^2)}$ with an adjacency matrix (scanning a whole row per vertex).

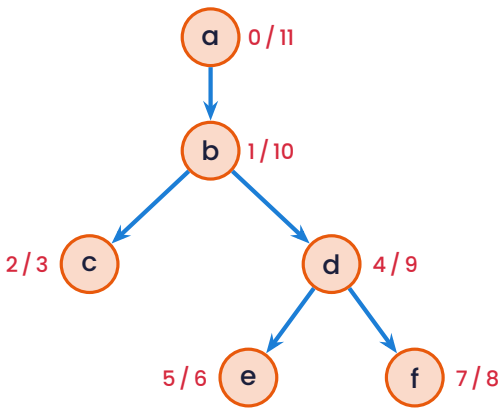
If the graph is disconnected, restart DFS from each unvisited vertex → a **DFS forest** (several trees). 📝 p. 46

DFS Parenthesis Theorem 📝 p. 48 — write "(u" when u starts and "u)" when u finishes; the result is always a **well-balanced bracket string** (like $(u(p(qq)p)(vv)u)$). **Consequence:** for any two vertices u, v **exactly one** holds:

1. Intervals [d(u), f(u)] and [d(v), f(v)] are **disjoint** → neither is a descendant of the other.
2. [d(u), f(u)] is **inside** [d(v), f(v)] → u is a **descendant** of v.
3. The reverse nesting → v is a descendant of u. **Partially overlapping intervals are impossible.**

> ⚡ **GATE Shortcut (interval questions, 📝 p. 49):** Given intervals like v_1:[4,5], v_2:[7,8], v_3:[12,13], v_4:[14,15], v_5:[3,6], v_6:[2,9], v_7:[1,10], v_8:[11,16], sort by start time and nest them: v_7[1,10] ⊃ v_6[2,9] ⊃ v_5[3,6] ⊃ v_1[4,5]; v_6 ⊃ v_2[7,8]; v_8[11,16] ⊃ v_3[12,13], v_4[14,15]. That gives a **forest of 2 trees** (roots v_7 and v_8). Any pair with *crossing* intervals (e.g. [1,5] and [3,8]) is **not** a valid DFS output.

White-path theorem (🆕): v becomes a descendant of u iff at time d[u] there is a path from u to v consisting only of **unvisited (white)** vertices.



Each label is start / finish. Counter ticks at every start and every finish.

Figure 4.2. DFS tree with discovery/finish times from source a (lecture-notes example).

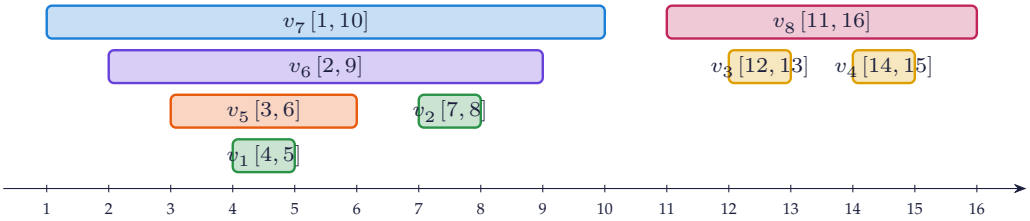


Figure 4.3. DFS intervals are perfectly nested or disjoint (never crossing): here $v_7 \supset v_6 \supset v_5 \supset v_1$, $v_6 \supset v_2$ and $v_8 \supset v_3, v_4$ — a forest of 2 trees.

4.4 Edge classification

📝 Lecture notes pp. 50–52

After DFS, every edge is one of four kinds. For a directed edge u → v:

Type	Meaning	Timestamp test
Tree edge (TE)	edge used to discover v	v first discovered via u
Forward edge (FE)	ancestor → (non-child) descendant, not a tree edge	d[u] < d[v] < f[v] < f[u]
Back edge (BE)	descendant → ancestor (**self-loops count as back edges**)	d[v] < d[u] < f[u] < f[v]
Cross edge (CE)	between vertices with no ancestor relation	f[v] < d[u] (v finished before u started)

Worked example (standard 6-vertex graph; same idea as notes p. 51): edges u→v, u→x, v→y, x→v, y→x, w→y, w→z, z→z. DFS from u (alphabetical), then w.

```
u 1/8 → v 2/7 → y 3/6 → x 4/5       w 9/12 → z 10/11
Edges:  u→v TE   v→y TE   y→x TE   x→v BACK (v is an ancestor, still open)
        u→x FORWARD (x is a descendant of u, not a tree edge)
        w→y CROSS (y finished at 6 < 9, no ancestry)    w→z TE    z→z BACK (self-loop)
```

Undirected graphs 📝 pp. 51–52: only **tree edges and back edges** exist. *Why no cross edges?* If an edge joined two unrelated vertices, DFS would have walked along it and made it a tree edge. Also any edge you might call "forward" can equally be seen as a back edge from the other side.

> ⚠️ **Trap:** The classification **depends on the order** in which DFS picks neighbours. The same graph can give different edge labels with different traversal orders (📝 p. 51 note). Questions normally fix the order (alphabetical/numeric) or give the d/f times.

> ⚡ **Memory hook:** *Tree and Forward go down; Back goes up; Cross goes sideways (to the left, i.e. to an earlier-finished vertex).*

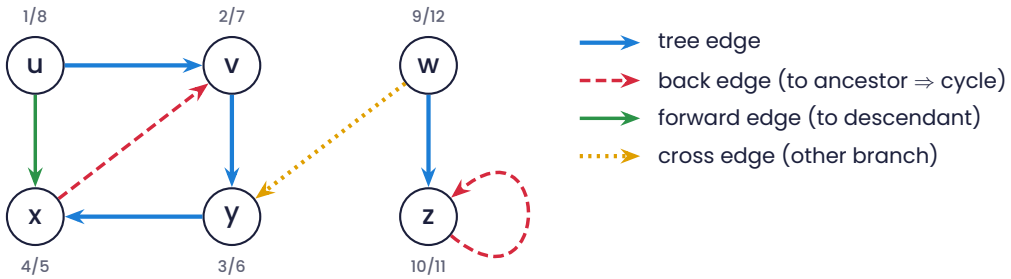


Figure 4.4. Edge classification on a directed graph (start/finish times shown). A directed graph has a cycle iff *DFS finds a back edge.*

4.5 DFS application 1 — Cycle detection

📝 Lecture notes pp. 52–53

Theorem: A graph has a cycle **iff** a DFS produces a **back edge** (works for directed and undirected).

- ***Back edge ⇒ cycle:*** tree edges give a path ancestor → descendant; the back edge closes it.
- ***Cycle ⇒ back edge:*** let v_0 be the first cycle vertex DFS reaches. Every other cycle vertex becomes its descendant; the cycle vertex that points back to v_0 gives an edge to an *ancestor* = back edge.
- **Counting:** if DFS finds n back edges, the graph contains **at least n cycles** (each back edge closes a distinct cycle).

Practical rules

- **Undirected:** a cycle exists iff while exploring u you meet an **already-visited neighbour that is not u's parent**.
- ⚠️ This rule is **wrong for directed graphs** (a visited vertex may just be a cross/forward edge target, e.g. a "diamond" a→b→d, a→c→d has no cycle). In a directed graph you need a vertex **currently on the recursion stack** (grey), i.e. a back edge.

4.6 DFS application 2 — Topological sort

📝 Lecture notes pp. 54–55

Topological order of a DAG: a line-up of vertices so that **every edge u→v has u before v** (like a prerequisite list).

- **Exists iff the graph is a DAG** (any cycle makes it impossible).
- **Method:** run DFS; list vertices in **decreasing order of finish time**. Reason: for every edge u→v in a DAG, **f[u] > f[v]** (v always finishes first, whether u or v is started first — notes p. 54 show both scenarios).
- A DAG always has **at least one vertex with in-degree 0** (a source).
- A DAG can have **many** topological orders.
- **Kahn's algorithm (🆕, same $\Theta(V+E)$):** repeatedly remove an in-degree-0 vertex and decrease its neighbours' in-degrees.

⚡ **Shortcuts**

- A DAG has a **unique** topological order **iff** consecutive vertices of its order are joined by edges (i.e. it contains a Hamiltonian path).
- To count orders in small DAGs, think "place sources first, multiply the choices" — e.g. a→b, a→c, b→d, c→d has 2 orders (abcd, acbd).
- Edge u→v in a DAG ⇔ f[u] > f[v] (a favourite MSQ statement).

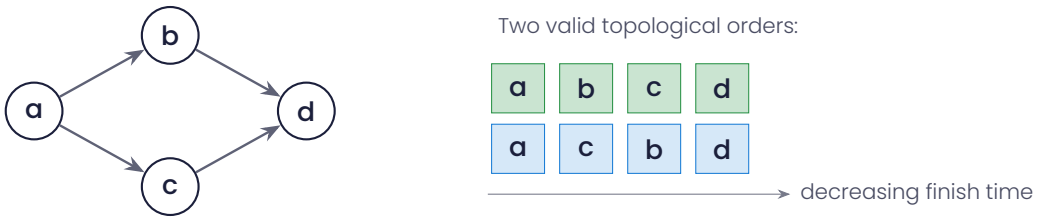


Figure 4.5. A DAG can have many topological orders; every edge points left-to-right in each of them.

4.7 DFS application 3 — Articulation points (cut vertices)

📝 Lecture notes pp. 55–57

An **articulation point (AP)** is a vertex whose removal **disconnects** the (undirected) graph.

- Brute force: remove each vertex and run DFS → O(V·(V+E)).
- **DFS rules** (single traversal, Θ(V+E)):
  1. **Root** of the DFS tree is an AP **iff it has ≥ 2 children** (subtrees can't be linked: undirected DFS has no cross edges).
  2. A **non-root** vertex u is an AP **iff some child subtree of u has no edge (back edge) going to a proper ancestor of u** — i.e. that subtree would be cut off.
  3. A **leaf** of the DFS tree is **never** an AP.

⚠️ **Trap (📝 p. 56):** the naive rule "u is an AP iff no descendant–ancestor edge exists" **fails**. Counter-example: vertex 6 is an AP although another descendant of 6 (vertex 10) has an edge to 6's ancestor — only the **subtree containing vertex 7** is isolated. The condition needs **at least one** child subtree that is cut off, not *all* of them.

The low-link formula (🆕 the quick mechanical test). Let `disc[v]` = discovery number; `low[v]` = smallest disc reachable from v's subtree using tree edges downward plus **at most one** back edge.

- Non-root u is an AP iff it has a child v with **low[v] ≥ disc[u]**.
- Edge (u,v) is a **bridge** iff **low[v] > disc[u]**.

Worked example: triangle 1-2-3, bridge 3–4, triangle 4-5-6. DFS from 1: disc = 1,2,3,4,5,6. `low[6] = 4` (back edge 6–4), `low[5] = 4`, `low[4] = 4`, `low[3] = 1` (via 3–1), `low[2] = 1`.

- u=3, child 4: low[4]=4 ≥ disc[3]=3 → **3 is an AP**; and 4 > 3 → **3–4 is a bridge**.
- u=4, child 5: low[5]=4 ≥ disc[4]=4 → **4 is an AP**.
- Root 1 has only 1 child (2) → not an AP. **APs = {3, 4}**.

📝 p. 56 example: *"On running DFS on G, v is found at a leaf position. Run DFS again with v as root. How many subtrees does v have?"* → **1**, because v can never be an AP (a leaf is never an AP, so removing v keeps the graph connected, so v as root has a single child subtree).

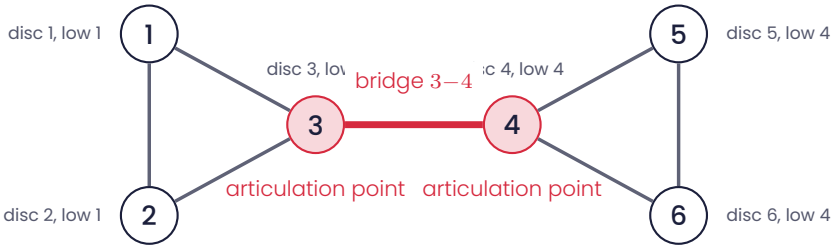


Figure 4.6. *Triangle–bridge–triangle: u is an AP if some child v has* $\text{low}[v] \geq \text{disc}[u]$*; (u, v) is a bridge if* $\text{low}[v] > \text{disc}[u]$.

4.8 Strongly Connected Components (SCC)

ADDED

A **strongly connected component** of a directed graph is a **maximal** set of vertices in which every vertex can reach every other. Shrinking every SCC to a single node gives the **condensation graph**, which is always a **DAG**.

Kosaraju's algorithm ($\Theta(V+E)$):

1. Run DFS on G and record **finish times**.
2. Build the **transpose** G^T (reverse every edge).
3. Run DFS on G^T, starting vertices in **decreasing order of finish time** from step 1. Each DFS tree of this second pass is one SCC.

Example: edges 1→2, 2→3, 3→1, 3→4, 4→5, 5→6, 6→4 → SCCs **{1,2,3}** and **{4,5,6}** (2 SCCs, and the condensation has a single edge {1,2,3}→{4,5,6}).

> ⚡ **GATE facts**
>
> - Minimum edges to add to make a DAG-of-SCCs strongly connected = **max(#source-SCCs, #sink-SCCs)** (when more than one SCC exists).
> - A graph is a DAG iff **every SCC is a single vertex**.
> - All vertices on a directed cycle belong to the same SCC.

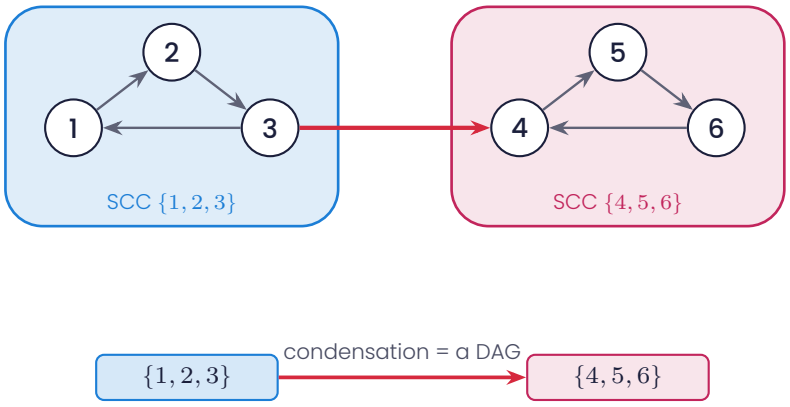


Figure 4.7. Strongly connected components and the condensation graph (always a DAG).

4.9 Breadth-First Search (BFS)

Lecture notes pp. 57–60

Idea: explore **all neighbours first**, then neighbours-of-neighbours — spreading out like ripples in a pond. Uses a **queue** (**FIFO**).

PSEUDOCODE

```
BFS(G, src):
    queue = [src]; visited[src] = true
    while queue not empty:
        node = dequeue(queue); process(node)
```

```
        for each neighbour v of node:
            if !visited[v]: visited[v] = true; enqueue(v)
```

Complexity: $\Theta(V + E)$ with AL, $\Theta(V^2)$ with AM — same as DFS. **Shape:** DFS trees are **long and skinny**, BFS trees are **short and fat** (notes p. 57).

Shortest paths: in an **unweighted** graph (or one where every edge has the same weight) BFS gives the **shortest path** (fewest edges) from the source to every vertex. (A weighted edge of weight w could be replaced by w unit edges, but then the cost depends on weights — hence Dijkstra, Ch 5.)

Queue state facts 📝 p. 58 — Think of BFS as exploring *layers* (level 0 = source, level 1 = its neighbours, ...):

- Only the current **frontier** (vertices discovered but not yet expanded) can be in the queue; **internal nodes are never in the queue**.
- Level i+1 vertices are enqueued only after *all* level i vertices are dequeued. At any time the queue holds vertices from **at most two consecutive levels**.
- Example (notes): tree A → {B,C,D}, B → {E,F}, E → {G,H,I}... if G,H,I (level 3) are already discovered then C and D can't be in the queue any more; only F, G, H, I can be.

Which edges can exist between BFS levels? 📝 pp. 58–59

Question	Undirected	Directed
Edge between **non-adjacent levels (skip-level)** going *down* (forward edge)	❌ impossible (it would have been explored earlier)	❌ impossible (same reason)
Edge going **up many levels** (back edge)	n/a (undirected has no direction)	✅ possible (back edge)
Edge **inside the same level**	✅ (cross edge)	✅ (cross edge)
Edge between **two consecutive levels** (not a tree edge)	✅ (cross edge)	✅ (cross edge)

BFS edge types 📝 p. 60

- **Directed:** tree edge, back edge, cross edge (cross edges are either same-level or between consecutive levels).
- **Undirected:** tree edge and cross edge only (same-level or consecutive-level).

Cycle parity (undirected): a cross edge **within the same level** closes an **odd-length** cycle; a cross edge **between consecutive levels** closes an **even-length** cycle.

BFS application 1 — Bipartite test 📝 pp. 60–61 A graph is **bipartite** (vertices can be split into two groups with all edges between the groups; equivalently 2-colourable) **iff it has no odd-length cycle**. Trees are always bipartite.

- Run BFS. If there is a **same-level edge ⇒ odd cycle ⇒ NOT bipartite**.
- If there are no cross edges, or only edges between consecutive levels (even cycles), it **is** bipartite. (Colour by level parity.)

BFS application 2 — Number of connected components (undirected; BFS or DFS): run BFS/DFS from every unvisited vertex; the **number of trees in the BFS/DFS forest = number of components**. $\Theta(V+E)$.

> ⚡**GATE 2018 fact (🆕):** If an undirected connected graph has a spanning tree T that is **simultaneously a DFS tree and a BFS tree** (same root), then **G = T** (G has no non-tree edges). Reason: any extra edge would be a back edge in DFS (joining ancestor–descendant) but a same-level/adjacent-level cross edge in BFS — it cannot be both.

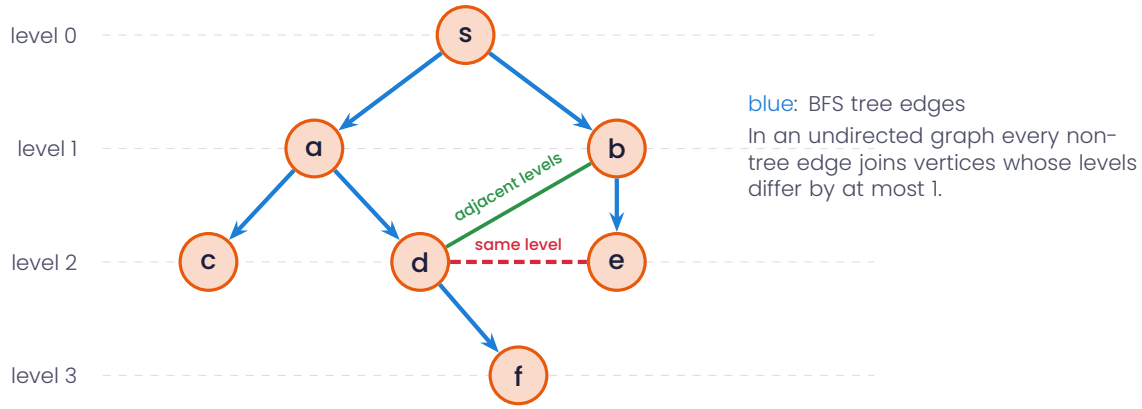


Figure 4.8. BFS layers: the tree is short and fat; the level of a vertex is its shortest hop-distance from the source.

4.10 DFS vs BFS cheat-sheet

	DFS	BFS
Data structure	stack / recursion	queue
Time (AL)	$\Theta(V+E)$	$\Theta(V+E)$
Space	$O(V)$ (stack depth up to V)	$O(V)$ (frontier up to V)
Finds shortest paths (unweighted)	❌	✅
Detects cycles	✅ (back edge)	✅ (undirected: same/adjacent-level non-tree edge)
Topological sort	✅	✅ (Kahn)
Articulation points, bridges, SCC	✅	❌
Bipartite check, components	✅	✅
Edge types	directed: TE, FE, BE, CE; undirected: TE, BE	directed: TE, BE, CE; undirected: TE, CE

4.11 Practice — Chapter 4

🎯 TRY IT YOURSELF -- cover the answers first

1. **Edge u→v in a directed graph has d[v] < d[u] < f[u] < f[v]. → Back edge.**
2. **Can an undirected DFS have a cross edge? → No.**
3. **How many topological orders: a→b, a→c, b→d, c→d? → 2.**
4. **Undirected connected graph, n vertices, n−1 edges. Bipartite?** → It's a tree → **Yes**.
5. **BFS from s: can an undirected edge join a level-1 vertex and a level-3 vertex? → No.**
6. **Number of APs in: triangle(1,2,3) – edge 3–4 – triangle(4,5,6)? → 2** (vertices 3 and 4).
7. **Directed graph 1→2, 2→1, 2→3, 3→4, 4→3: # SCCs? Min edges to make it strongly connected?** → SCCs {1,2},{3,4} = **2**; one source SCC & one sink SCC → **1** edge (e.g. 4→1).
8. **Max edges in a DAG on 6 vertices?** → $6 \cdot 5/2 = \mathbf{15}$.
9. **Cycle detection by "visited non-parent neighbour" on a directed graph — valid? → No**; use a back edge to a vertex on the stack.
10. **MSQ:** In a DFS of a *directed* graph which hold? (a) f[u] > f[v] for every edge u→v of a DAG ✓ (b) a graph is acyclic iff no back edge ✓ (c) cross edges point to vertices that started earlier **and** finished earlier ✓ (d) every edge u→v satisfies d[u] < d[v] ✗ (back/cross edges violate it).
11. **Time to find the number of connected components with AM? →** $\Theta(V^2)$.
12. **BFS tree has depth 5 from s. What is the shortest-path length from s to a deepest vertex (unweighted)? → 5**.

CHAPTER 5

Shortest Paths: Greedy Thinking, Dijkstra, Bellman-Ford, Floyd-Warshall

5

📝 Notes pp. 61–80 and p. 116. Shortest-path questions appeared in 2020, 2025 (×2), 2026 and are the 2-mark favourites: **trace a table, say which algorithm works, or give a complexity.**

5.1 What is a ``greedy'' algorithm?

📝 Lecture notes p. 61

A **greedy** algorithm builds a solution step by step, always taking the **best-looking choice right now** and never undoing it.

- It always produces a *feasible* answer, but is **not guaranteed optimal**.
- It *can* be proven optimal when the problem has both:
  1. **Optimal substructure** — an optimal solution contains optimal solutions of its sub-problems.
  2. **Greedy-choice property** — a locally best choice can be part of a globally optimal solution.

5.2 The Single-Source Shortest Path (SSSP) problem

📝 Lecture notes pp. 61–63

Given a weighted graph and a source s, **find the cheapest path from s to every other vertex.**

How a beginner would try, and why each fails or works:

Approach	Verdict
1. List all paths, pick the cheapest	correct but **exponential** time
2. Join two shortest sub-paths	doesn't work in general
3. BFS after replacing weight-w edges by w unit edges	works but cost depends on weights (e.g. weight 30 → 30 extra nodes)
4. **Greedy "finalise the closest vertex"**	**this is Dijkstra** ✅ (positive weights)

Lemma (optimal sub-paths, p. 62): *Any sub-path of a shortest path is itself a shortest path between its endpoints.* (If a shorter sub-path existed, swapping it in would shorten the whole path.)

Why Dijkstra's greedy choice is safe (p. 62): suppose from S the known edges give distances S→A = 1, S→B = 2, S→C = 3. Can any route reach A faster than 1? Any other route first pays at least 2

(via B) or 3 (via C) and then more (all weights positive), so **the smallest known distance is final.** Finalise it, relax its out-edges, repeat.

5.3 Dijkstra's algorithm (non-negative weights)

📝 Lecture notes pp. 62–68

Relaxation: to *relax* edge (u,v) means: `if d[u] + w(u,v) < d[v] then d[v] = d[u] + w(u,v)` (and remember parent[v] = u).

Algorithm

PSEUDOCODE

```
1. d[source] = 0, d[every other vertex] = ∞;  put all vertices in a priority queue keyed by d.
2. Repeat until the queue is empty:
     u = Extract-Min(queue)          ← u's distance is now FINAL
     for every edge (u, v) with v not yet finalised: Relax(u, v)
```

PSEUDOCODE

```
DIJKSTRA(G, src):
    seen = []; pq = all vertices with ∞;  pq[src] = 0
    while pq not empty:
        curr = pq.dequeue();  seen[curr] = true
        for v in Adj[curr]:
            if !seen[v]: RELAX(curr, v, G)
```

Worked example 📝 p. 65 (source U). Edges (as read from the figure): U–M 12, U–CS 3, U–L 2, U–S 21, CS–M 4, CS–N 15, L–P 2, P–N 10, P–O 1, O–S 15.

Finalised	U	M	CS	L	P	N	S	O
start	0	∞	∞	∞	∞	∞	∞	∞
U (0)	–	12	3	2	∞	∞	21	∞
L (2)	–	12	3	–	4	∞	21	∞
CS (3)	–	7	–	–	4	18	21	∞
P (4)	–	7	–	–	–	14	21	5
O (5)	–	7	–	–	–	14	20	–
M (7)	–	–	–	–	–	14	20	–
N (14)						–	20	
S (20)							–	

Shortest-path tree: U→L→P→O→S, U→CS→M, P→N. Final distances: U0, L2, CS3, P4, O5, M7, N14, S20. ✓ (verified by code) Each row: pick the smallest un-finalised number, mark it final, then update the others via it. **Update rule used:** `d[K] = min(d[K], d[M] + w(M,K))`.

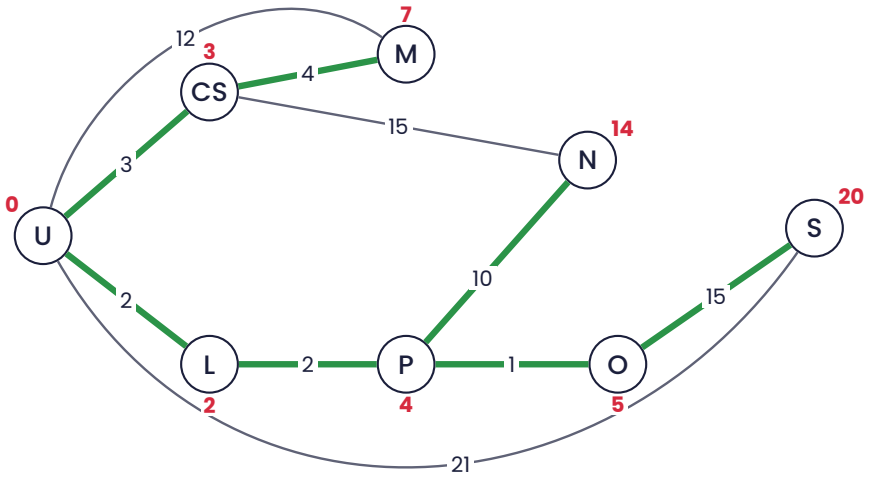


Figure 5.1. Dijkstra from U: red numbers are final distances, thick green edges form the shortest-path tree (distance to S is $5 + 15 = 20$, not the direct edge of 21).

5.3.1 The famous GATE ``PQ + parent table'' question

Lecture notes pp. 66–67

Source = 0. The table shows the priority queue values and parents ***immediately after vertex 4 has been deleted from the PQ and relaxed****:*

v	0	1	2	3	4	5	6	7
PQ value	0	2	13	23	11	7	36	19
parent	null	0	5	0	5	1	5	4

Edge 4→7 has weight x, edge 5→7 has weight y. Which condition on x, y is forced? Options: (A) x = 8 and y ≥ 12 (B) x > 8 and y = 11 (C) x = 7 and y = 11 (D) x > 8 and y = 12.

Method — replay the history of vertex 7:

1. Vertex 5 (dist 7) was finalised earlier: relaxing 5→7 gave **d[7] = 7 + y**.
2. Vertex 4 (dist 11) was then chosen *before* $7 \Rightarrow 11 \leq 7 + y$ (so $y \geq 4$, no help).
3. After relaxing 4→7 the final value is **19 with parent 4** ⇒ the update *came from 4*: $11 + x = 19 \Rightarrow x = 8$, and it *beat* the old value: $11 + x < 7 + y \Rightarrow 19 < 7 + y \Rightarrow y > 12$ (with a tie y = 12 the parent normally stays 5; the official key lists **y ≥ 12**).
4. Options B, C fail immediately (x ≠ 8). Option D (x > 8) would make 11 + x > 19, so 19 must have come from 5, parent 5 ✗.

Answer: (A) x = 8 and y ≥ 12. > ⚡ **Technique:** "after deleting vertex k" tables are solved by (i) finding which relaxation produced each final value (parent!), (ii) writing the equality for the parent route, and (iii) writing the inequality for the other routes.

5.3.2 Dijkstra with negative edges

📝 Lecture notes pp. 68–70

Dijkstra's assumption — "once finalised, no later path can be cheaper" — breaks when **negative edges** exist.

Example 1 — A –20→ B –(−10)→ C –3→ D: only one path; Dijkstra works (distances 0, 20, 10, 13). **Example 2** (works by luck): A –4→ B, A –2→ D, B –6→ C, C –(−3)→ D … Dijkstra still gives the right answer here. **Example 3 (fails):** A –4→ B, A –2→ D, B –6→ C, C –(−9)→ D. Dijkstra finalises D = 2 (smallest), but the path A→B→C→D costs 4+6−9 = **1** < 2 ⇒ **wrong. Example 4 (fails):** A –5→ B –(−10)→ C and A –3→ C. Dijkstra finalises C = 3, but A→B→C = **−5**.

"Fix" attempt (p. 71): add a constant to every edge to remove the negatives. Take A –5→ B –(−10)→ C and a direct edge A –3→ C. Originally A→B→C costs −5 (best) vs 3 direct. Add **10** to every edge: A→B 15, B→C 0, A→C 13 — now the direct edge (13) beats A→B→C (15). The *best path changed*, because a path with more edges pays the constant more times. ⇒ **No such trick works for shortest paths** (it *does* work for MST — see Ch 6).

Rules to remember

- Dijkstra **may or may not** give the right result with negative edges, and it **cannot detect a negative cycle**. Never use it when any weight is negative.
- A shortest path **never contains a cycle** (when no negative cycles exist): a positive cycle only adds cost; a *negative* cycle makes the "shortest path" undefined (you can loop forever, cost → −∞). 📝 p. 71

5.3.3 Time complexity of Dijkstra

📝 Lecture notes pp. 72–73

```
T = V × T(Extract-Min)  +  E × T(Relax / Decrease-key)          (each vertex extracted once, each
↪  edge relaxed once)
```

Graph repr.	Priority queue	Total time
Adjacency list	**Binary heap**	**$\Theta((V+E)\log V)$**
Adjacency matrix	Binary heap	$\Theta(V^2 + (V+E)\log V) = \Theta(V^2)$
Adjacency list	Unsorted array	$\Theta(V^2 + E) = \mathbf{\Theta(V^2)}$
Adjacency matrix	Unsorted array	$\Theta(V^2)$
Adjacency list	Sorted array	$\Theta(V + E \cdot V) = \Theta(EV)$
Adjacency list	**Fibonacci heap**	**$\Theta(V \log V + E)$** (the best)

⚡ **Which to choose?** Sparse graph ($E \approx V$): heap is best — $\Theta(V \log V)$. Dense graph ($E \approx V^2$): the plain array version $\Theta(V^2)$ beats the heap's $\Theta(V^2 \log V)$. **The "default" answer** to "complexity of Dijkstra" is **$O((V + E) \log V)$** (adjacency list + binary heap) — and $O(E + V \log V)$ with a Fibonacci heap.

5.4 Shortest path in a DAG (works with negative edges!)

📝 Lecture notes pp. 74–75

Key idea: if you process vertices in **topological order**, then by the time you reach a vertex *all of its incoming edges have already been relaxed*, so its distance is final.

```
1. Topologically sort the DAG.
2. d[s] = 0, others ∞.
3. For each vertex u in topological order: relax every outgoing edge of u.
```

Time $\Theta(V + E)$ (topological sort + one relaxation per edge). **Works with negative weights.**

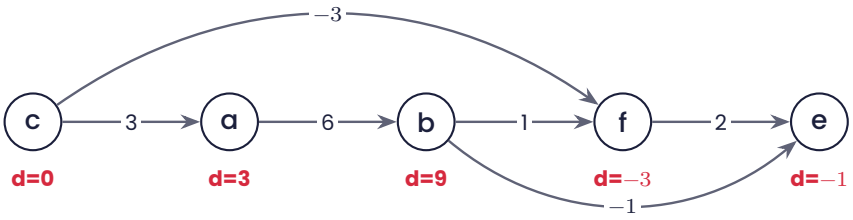
Example 1 📝 p. 74: s –10→ a –5→ b and s –7→ b. In order s, a, b: a = 10; b = min(7, 10+5) = **7**.

Example 2 📝 p. 75 (source c): edges c→a (3), c→f (−3), a→b (6), b→f (1), b→e (−1), f→e (2).

Processed	a	b	f	e
c (0)	3	∞	−3	∞
a (3)	–	9	−3	∞
b (9)	–	–	−3	8
f (−3)	–	–	–	−1
e (−1)				

Final: a = 3, b = 9, f = −3, **e = −1** (via c→f→e).

> ⚡Shortest path in DAG is **faster than Dijkstra** ($\Theta(V+E)$) *and* handles negative edges. **Longest path in a DAG** (🆕) uses the same method with max instead of min (or negate the weights) → $\Theta(V+E)$. Longest path in a *general* graph is NP-hard.



vertices in topological order; relax edges left to right, one pass, $\Theta(V + E)$

Figure 5.2. Shortest paths in a DAG with source c: handles negative weights without Dijkstra or Bellman–Ford.

5.5 Bellman–Ford algorithm (negative edges allowed)

📝 Lecture notes pp. 76–80

How the idea is born (notes p. 76–77): Suppose someone secretly tells you the shortest path S → A → C → E → T. You'd just relax edges *in that order* and be done. You don't know the path, but you can **relax edges in the path's order while also relaxing everything else** — extra relaxations never hurt. So relax **all edges, in any fixed order, again and again**: after one pass, at least the *first* edge of every shortest path is in order; after two passes the first two; and so on.

Bellman–Ford rule: *Relax **all E edges** (in any order) **V − 1** times. Then every shortest path is guaranteed to have been realised.* (A shortest path has at most V − 1 edges, so V − 1 passes suffice.)

After k passes you have the correct distance to every vertex whose shortest path uses **at most k edges** (p. 80).

Algorithm

PSEUDOCODE

```
BELLMAN-FORD(G, s):
    d[s] = 0, others ∞
    repeat V - 1 times:          for each edge (u,v): relax(u,v)
    one more pass:               for each edge (u,v): if d[u] + w < d[v] → NEGATIVE CYCLE
    ↪ reachable from s
```

Negative-cycle detection (the V-th pass): if **any** distance still changes in the extra pass, a negative cycle exists. If nothing changes, the answers are final.

Early exit (p. 79): if a whole pass changes nothing, nothing will ever change again → stop.

Time: (V−1)·E + E = **Θ(V·E)**.

Worked examples (all from notes)

Example 1 (p. 77–78): edges A→B 1, A→C 10, C→B −1, B→D 5, D→C 2; order **A→C, A→B, D→C, C→B, B→D**; V−1 = 3 passes.

	A	B	C	D
init	0	∞	∞	∞
pass 1	0	1	10	6
pass 2	0	1	8	6
pass 3	0	1	8	6

Final: B=1, C=8, D=6. (Cycle B→D→C→B has weight 5+2−1 = 6 > 0, so no negative cycle.)

Example 2 (Dijkstra fails here): A→B 4, A→C 2, B→C −5. Order **B→C, A→C, A→B**, V−1 = 2. pass 1: C=2, B=4 → pass 2: C = 4−5 = **−1**. ✓ (Dijkstra would say 2.)

Example 3 — negative cycle (p. 79): A→B 1, A→C 10, B→D 5, D→C 2, C→B −10 (cycle B→D→C→B weighs 5+2−10 = **−3**). Order **C→B, B→D, D→C, A→B, A→C**, V−1 = 3.

	A	B	C	D
init	0	∞	∞	∞
1	0	1	10	∞
2	0	0	7	5
3	0	−3	4	2
extra	0	**−6**	**1**	**−1**

Values still change in the extra pass ⇒ **negative cycle detected.**

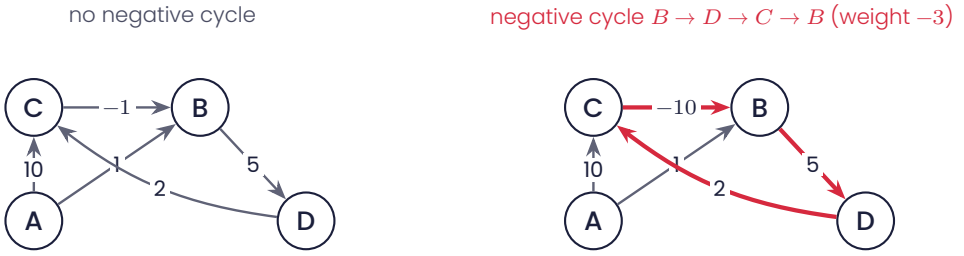


Figure 5.3. Bellman–Ford: relaxing all edges $V - 1$ times works on the left; on the right the V-th round still improves some distance, exposing a negative cycle.

5.5.1 Dijkstra vs Bellman–Ford

📝 Lecture notes p. 80

	Dijkstra	Bellman--Ford
Time	O((V+E) log V)	O(V·E)
Dense graph (E ≈ V²)	V² log V	V³ → Dijkstra better
Sparse graph (E ≈ V)	V log V	V² → Dijkstra better
Negative edges	❌ may fail	✅ works
Negative-cycle detection	❌	✅
Strategy	Greedy	Dynamic programming–like

5.6 All-pairs shortest paths: Floyd–Warshall

📝 Lecture notes p. 116

- Gives the shortest distance between **every pair** of vertices.
- **Time Θ(V³)**, space Θ(V²). Works with **negative edges** (no negative cycles).
- **DP recurrence:** `d_k[i][j] = min( d_{k-1}[i][j], d_{k-1}[i][k] + d_{k-1}[k][j] )` — allow vertex k as an intermediate stop, for k = 1…V (three nested loops, k outermost).
- **Negative-cycle test:** after the algorithm, **a negative number on the diagonal** (d[i][i] < 0) means a negative cycle.

> ⚡Choosing an all-pairs method: $V \times$ Dijkstra $= O(V(V+E)\log V)$ is better for sparse graphs with non-negative weights; Floyd–Warshall $\Theta(V^3)$ is simplest for dense graphs.

5.7 Travelling Salesman (TSP)

📝 Lecture notes p. 116

Visit every city exactly once and return, with minimum total cost. Brute force tries $(n-1)!$ tours. **No polynomial-time algorithm is known** (NP-hard); the DP (Held–Karp) is $\Theta(n^2 \cdot 2^n)$.

5.8 Shortest-path cheat-sheet

Situation	Use	Time
Unweighted	BFS	$\Theta(V+E)$
Non-negative weights	Dijkstra	$O((V+E)\log V)$
Negative edges, no negative cycle	Bellman–Ford	$O(VE)$
DAG (any weights)	Topological order + relax	$\Theta(V+E)$
All pairs	Floyd–Warshall	$\Theta(V^3)$
Detect negative cycle	Bellman–Ford (extra pass) / Floyd (diagonal)	

5.9 Practice — Chapter 5

🎯 TRY IT YOURSELF -- cover the answers first

1. **Dijkstra on A–B 4, A–C 1, C–B 2, B–D 1, C–D 5 (undirected), source A.** → A0 → C1 → B3 (via C) → D4 (via B); $\mathbf{d(D) = 4}$.
2. **Which algorithm for a graph with negative edges but no negative cycle, V=1000, E=3000?** → Bellman–Ford ($O(VE) = 3 \cdot 10^6$).
3. **Dijkstra with adjacency matrix & array PQ?** → $\Theta(V^2)$.
4. **Bellman–Ford: shortest path to t has 3 edges. Minimum number of full passes guaranteed (any order)?** → **3**.
5. **Statement: "If all edge weights are distinct, the shortest path between two nodes is unique."** → **False** (two different paths can have equal total).
6. **Statement: "Adding 10 to every edge never changes the shortest path tree."** → **False** (paths with more edges are penalised more).
7. **Statement: "Squaring/adding a constant to every edge weight never changes the *MST*."** → **True** (Ch 6).
8. **A graph has a negative cycle reachable from s. Bellman–Ford after V−1 passes + 1 extra pass?** → at least one distance **still decreases**.
9. **Complexity of shortest path in a DAG?** → $\Theta(V+E)$.

10. **Dijkstra on a graph with a negative edge: which is guaranteed?** → *Nothing* — it may output a wrong distance; it will still terminate.

CHAPTER 6

Minimum Spanning Trees

MST

📝 Notes pp. 81–98. MST is worth **1.56 marks/paper**; "number / weight of MSTs" NAT questions appeared in 2018, 2020, 2021, 2022, 2024, 2025 — every one is a pure practice problem.

6.1 What is a spanning tree?

📝 Lecture notes p. 81

A **spanning tree** of a connected undirected graph is a subgraph that

1. contains **all V vertices**,
2. is **connected**, and
3. has **no cycle** (⇒ exactly **V − 1 edges**).

Example (K_4 = 4 vertices, all 6 edges): choose any 3 edges *except* the 4 triples that form a triangle → $C(6,3) - 4 = 20 - 4 =$ **16 spanning trees** ($= 4^{4-2}$ by **Cayley's formula: K_n has n^{n-2} spanning trees**). Enumerating all trees is hopeless for big graphs → we need clever algorithms.

With **edge weights**, the spanning tree of **least total weight** is the **Minimum Spanning Tree (MST)**. 📝 p. 81 example: square A–B (10), B–C (40), C–D (30), D–A (20). A spanning tree = delete one edge; delete the heaviest (40) → **MST weight 10 + 20 + 30 = 60.**

6.2 The two golden properties

📝 Lecture notes pp. 82–84

Cut: a way of splitting the vertices into two non-empty groups. A graph with n vertices has $\mathbf{2^{n-1} - 1}$ cuts (2^n subsets, halved because $(S, \bar{S}) = (\bar{S}, S)$, minus the empty/everything split). An edge **crosses** the cut if its endpoints lie in different groups.

Cut Property: *For any cut, the **minimum-weight edge crossing it** belongs to an MST.*

- If that minimum edge is **unique**, it is in **every** MST.
- If several crossing edges tie for the minimum, **each** of them is in *some* MST (but not necessarily all).

Cycle Property: *In any cycle, the **maximum-weight edge** is not needed.*

- The **heaviest edge of a cycle** can't be in an MST (if it is the *unique* heaviest).
- **An edge e is in no MST ⟺ it is the unique heaviest edge of some cycle.**

- If several edges tie for heaviest in a cycle, at least one of them is excluded.

Bridge rule: an edge whose removal disconnects the graph (a **bridge**) is in **every** spanning tree, hence in every MST.

> ⚡**Distinct weights ⇒ the MST is unique.** (Ties are what create multiple MSTs.)

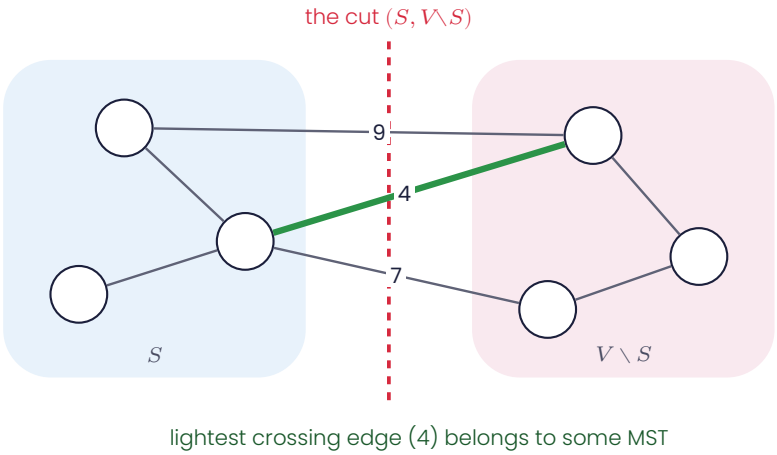


Figure 6.1. Cut property: for any cut, the minimum-weight edge crossing it is safe to add to a minimum spanning tree.

6.3 Kruskal's algorithm

📝 Lecture notes pp. 83–85

Idea: *Repeatedly take the cheapest remaining edge unless it would create a cycle.* (By the cut property each accepted edge is safe.)

PSEUDOCODE

```
KRUSKAL(G):
    T = {}
    sort edges by weight, ascending
    for each edge e = (u,v) in that order:
        if adding e to T creates no cycle:   T = T U {e}
```

Trace (use this graph repeatedly): A–B 10, B–C 12, B–D 9, A–D 5, D–E 7.

Edge (sorted)	Weight	Action
A–D	5	add
D–E	7	add
B–D	9	add
A–B	10	**skip** (A and B already connected via D)
B–C	12	add

MST = {AD, DE, BD, BC}, **weight 33**.

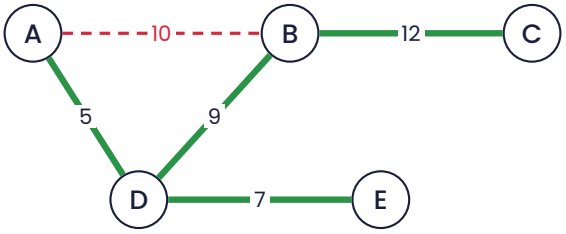
Cycle check with Union–Find: keep each component as a set; edge (u,v) makes a cycle iff `find(u) == find(v)`; otherwise `union`. Near-O(1) per operation.

Time complexity 📝 p. 85

- Sort: $\Theta(E \log E)$. Cycle check per edge: DFS → $O(V+E)$ each → total $O(E^2)$-ish; **Union–Find → almost $O(1)$** each.
- **Total: $\Theta(E \log E) = \Theta(E \log V)$** (since $E \le V^2$, $\log E \le 2 \log V$).

Two golden facts about Kruskal

1. It uses only the **order** of the weights, so it works with **negative weights**.
2. Adding/subtracting a **constant** to all weights, or applying any **order-preserving transform** (squaring positive weights, taking log, etc.), keeps the sorted order ⇒ **the MST does not change.** 📝 p. 93 question: *"Distinct positive weights, MST = T. Square every weight; is the MST still T?"* → **Yes** ($e_1 < e_2 < \ldots \Rightarrow e_1^2 < e_2^2 < \ldots$). *(Contrast: shortest paths DO change — Ch 5.)*



Kruskal order
5 (A--D) ✓
7 (D--E) ✓
9 (B--D) ✓
10 (A--B) × closes a cycle
12 (B--C) ✓
MST weight = 5 + 7 + 9 + 12 = 33

Figure 6.2. Minimum spanning tree (thick green) of a 5-vertex graph; the dashed red edge would form a cycle and is rejected.

6.4 Prim's algorithm

📝 Lecture notes pp. 87–90

Idea: *Grow a single tree from any start vertex; at each step add the **cheapest edge leaving the tree**.* (Every step is a cut — tree vs rest — so the cut property guarantees safety. A new vertex joins by one edge, so no cycle can form.)

Trace (same graph as above), starting at C:

Step	Tree	Cut-edges considered	Pick
1	{C}	B–C 12	**B–C (12)**
2	{B,C}	A–B 10, B–D 9	**B–D (9)**
3	{B,C,D}	A–B 10, A–D 5, D–E 7	**A–D (5)**
4	{A,B,C,D}	D–E 7	**D–E (7)**

MST weight = 12 + 9 + 5 + 7 = **33** (same MST as Kruskal ✓).

Implementation (priority queue of vertices keyed by cheapest connecting edge)

PSEUDOCODE

```
PRIM(G):
```

```
key[v] = ∞ for all v; pick any root r; key[r] = 0; parent[r] = null
pq = all vertices
while pq not empty:
    u = Extract-Min(pq)                 // u joins the tree
    for each neighbour v of u:
        if v in pq and w(u,v) < key[v]:  key[v] = w(u,v); parent[v] = u
```

Time: `V × T(Extract-Min) + E × T(Decrease-key)`

Representation	Priority queue	Time
Adjacency list	Binary heap	$\mathbf{O((V+E)\log V)} = O(E \log V)$
Adjacency matrix	Array	$\mathbf{O(V^2)}$
Adjacency list	Fibonacci heap	$\mathbf{O(E + V \log V)}$

⚡ **Which for what:** dense graph → Prim with array $O(V^2)$; sparse graph → Kruskal or Prim+heap $O(E \log V)$.

6.5 Prim vs Dijkstra, and MST ≠ shortest-path tree

Lecture notes pp. 91–92

- **Prim**: at each step add the vertex **closest to the current *tree***.
- **Dijkstra**: at each step add the vertex **closest to the *source***. Same code skeleton, different key: Prim's key = the *edge weight* w(u,v); Dijkstra's key = $d[u] + w(u,v)$. Both are $O(V \log V + E)$ with Fibonacci heaps.

The two trees can differ: triangle A–B 5, B–C 5, A–C 8.

- **MST** = {AB, BC}, weight **10**.
- **Shortest-path tree from A** = {AB (5), AC (8)}, total **13** (the direct edge A→C = 8 beats A→B→C = 10). Dijkstra minimises distance **from the source to each node individually**; Prim minimises the **total tree weight** (p. 91).

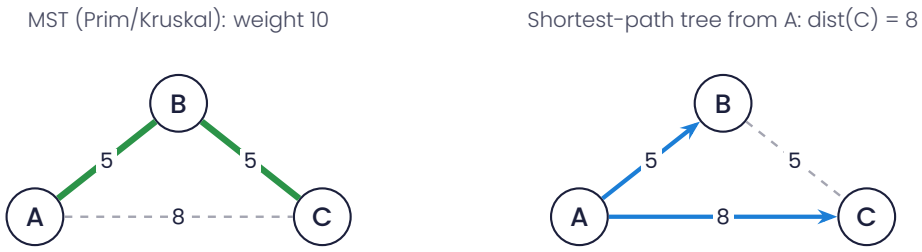


Figure 6.3. Prim and Dijkstra differ: Prim minimises the total weight *of the tree (10), Dijkstra minimises each* distance from A *(C at 8 rather than 10).*

6.6 Counting MSTs (the NAT favourite)

📝 Lecture notes pp. 85–87

The method (works for every such problem):

1. Edges that are **bridges** (or the unique cheapest crossing edge of some cut) are forced — fix them, they don't multiply the count.
2. Process the weights in **groups of equal weight** in increasing order. At a weight w, the earlier (cheaper) edges have already merged vertices into components. Count the number of ways to choose edges *of weight w* that connect those components **without forming a cycle**; **multiply the counts of all groups**.
3. A group where all weight-w edges join *different* pairs of components has exactly one way.

Examples

Graph	Reasoning	#MSTs
K_4, all weights equal	= number of spanning trees of K_4	**16**
4-cycle, all weights equal	drop any one of the 4 edges	**4**
Triangle, all weights equal	choose 2 of 3	**3**
Two triangles joined by a bridge (+ a pendant edge), all weights equal (📝 p. 86)	bridges/pendant forced; each triangle: $C(3,2) = 3$ choices	**3 × 3 = 9**
Four blobs R_1...R_4 with 4, 3, 3, 4 spanning trees each, joined by bridges, equal weights (📝 p. 86)	multiply spanning-tree counts of each blob	$4 \times 3 \times 3 \times 4 =$ **144**
Edges B–D = 1; A–B = 2, A–D = 2, A–C = 2	weight-1 edge forced → components {B,D}, {A}, {C}; need 2 of three weight-2 edges, but A–B and A–D both join A to {B,D} (pair would make a cycle) → $C(3,2) - 1$	**2**
Two triangles, one of weight 1 (edges AB,BC,AC) and one of weight 2 (CD,DE,CE) sharing vertex C	3 choices × 3 choices	**9** (MST weight 6)

> 📝 *Your notes (p. 86) wrote "149" for the four-blob example; $3^2 \times 4^2 = 144$ is the correct product.*

> ⚡ **Quick sanity for "all weights equal":** #MSTs = number of spanning trees of the whole graph. Useful numbers: triangle 3 · 4-cycle 4 · K_4 16 · K_5 125 · 5-cycle 5 · cycle C_n has n.

6.7 MST properties — true/false statements

📝 Lecture notes pp. 87, 93

1. *"The minimum-weight edge is in **some** MST."* → **True.** *"...in **all** MSTs."* → **False** (triangle with two weight-1 edges: either one can be used).
2. *"Distinct weights ⇒ unique MST."* → **True.**
3. *"Edge $e = (u,v)$ is in no MST **iff** u and v are joined by a path whose edges are all **strictly lighter** than e."* → **True.** If the path may contain edges **equal** to $w(e)$, then e can still belong to *some* MST (📝 p. 93 counter-example: u–v weight 3 with an alternative path of weights 3, 1, 3 — the tie lets e be swapped in).
4. *"The unique lightest edge crossing some cut is in every MST."* → **True** (cut property).
5. *"Maximum spanning tree"* → run Kruskal with edges sorted **descending** (or negate all weights).
6. **Minimum-bottleneck property** (🆕)**:** the path between any two vertices **inside the MST** minimises the **largest edge** over all paths between them.
7. A graph with $E = V - 1$ and connected is its own unique spanning tree.

6.8 ``Missing weight / range of x" problems

📝 Lecture notes pp. 94–98

Two rules solve every such question. Take the given MST T:

> **Rule A — tree edge with unknown weight x:** x must be **≤ the weight of every non-tree edge whose cycle (through T) contains it**. So **x ≤ min** of the covering non-tree edges. **Rule B — non-tree edge with unknown weight x:** x must be **≥ the heaviest tree edge on the T-path between its endpoints** (otherwise the cycle-property would swap it in).

📝 p. 94: figure with the blue MST; tree edge E–D labelled x. Non-tree edges C–D (9) and D–F (20) each form a cycle containing E–D ⇒ **x ≤ 9** and **x ≤ 20** ⇒ **x ≤ 9**. 📝 p. 95: non-tree edge D–E labelled x with the other three tree edges around the cycle A–B–D–E ⇒ x must be **≥ the max edge on that tree path** (the notes find $x \geq 2$). 📝 pp. 95–96: **"In G a certain MST contains edges of weight X, Y, Z plus seven other edges"** (12-vertex grid, all other weights known). • Identify the other seven edges by **growing cuts**: take the cut {A} | rest → the cheapest edge at A (A–E) must be in; take {A,E} | rest → next cheapest, ... (cut property each time). • *Could X be 120?* No — for the cut isolating that vertex a cheaper edge (90) would have been chosen. • *Could Y be 55?* Yes — it was chosen over an 80 edge, $55 < 80$ is consistent. • *Largest possible Z if all weights distinct?* Z is a tree edge; in the cycle through it the other edges include a 120; Z cannot exceed 120 and cannot equal 120 (distinct) ⇒ **Z ≤ 119**. 📝 pp. 96–98: *"MST contains edges F–G and B–C with weights x and z. What are the maximal values?"* — apply Rule A to each: A–G (weight 1) and F–G (weight x) are both in the tree, so the direct edge A–F (11) would close a cycle A–G–F and must not be lighter than F–G ⇒ **x ≤ 11**; adding B–H (8) would close the cycle B–C–D–I–H, so B–C must not be heavier than 8 ⇒ **z ≤ 8**.

6.9 Maintaining an MST when one edge weight changes

📝 Lecture notes pp. 97–98

Given the current MST T of G, give an efficient update. Four cases:

Change	What happens	Time
① Tree edge e **decreases**	T stays an MST (it was good, now even better)	O(1)
② Non-tree edge e **decreases**	add e to T → creates **one cycle**; remove the **heaviest edge on that cycle** (find the path u→v in T with DFS, take its max)	O(V) (tree has V−1 edges)
③ Tree edge e **increases**	remove e from T → two components; scan all edges and take the **cheapest one crossing between the components** (DFS to label components + one pass over E)	O(V + E)
④ Non-tree edge e **increases**	nothing changes (it was already excluded; heavier only makes it worse)	O(1)

6.10 Practice — Chapter 6

🎯 TRY IT YOURSELF -- cover the answers first

1. **Spanning trees of K_5?** → $5^3 = 125$.
2. **Number of cuts in a graph on 5 vertices?** → $2^4 - 1 = 15$.
3. **MST of A–B 5, B–C 5, A–C 8 (weight) and number of MSTs?** → **10**, unique (8 is the unique heaviest on the cycle).
4. **All edges weight 1; graph = 5-cycle plus one chord (the chord splits it into a triangle and a 4-cycle). #MSTs?** → count spanning trees by deletion–contraction on the chord e: $\tau(G) = \tau(G - e) + \tau(G / e) = 5$ (the plain 5-cycle) + 6 (contracting e leaves a doubled edge, 2 trees, glued to a triangle, 3 trees: 2×3) = **11** (verified by brute force).
5. **Kruskal's complexity with sorted-edge input and Union-Find?** → $O(E \cdot \alpha(V)) \approx \mathbf{O(E)}$.
6. **A graph has all distinct weights. Does Prim started at different vertices give different trees?** → **No** — unique MST.
7. **Edge weight of the lightest edge of a cycle — in MST?** → not necessarily forced; only the **heaviest (unique)** is excluded.
8. **Which can handle negative weights: Kruskal / Prim / Dijkstra?** → Kruskal ✓, Prim ✓, Dijkstra ✗.
9. **Prim with adjacency matrix, array PQ?** → $\mathbf{O(V^2)}$.
10. **G has V vertices, E edges (connected). Min #edges to delete to get a spanning tree?** → **E − V + 1**.

PART

IV

Hashing

🟠 0.67 markspaper, 6 of 9 years

IN THIS PART

Ch 7 Hashing

CHAPTER 7

Hashing

🆕 NEW • not in your lecture notes

> **This chapter is not in your lecture notes.** The PYQ analysis shows hashing in 2020, 2021, 2022, 2023, 2025 and twice in 2026 (insert-and-trace NATs plus "simple-uniform hashing" expectations). So I wrote it from scratch, with fully traced examples.

7.1 The problem hashing solves

We want a dictionary: **insert, search, delete** keys *very* fast. Arrays give O(1) access but only by index; sorted arrays give O(log n) search but slow insert; linked lists give O(n) search. **Hashing** turns a key into an array index with a **hash function** $h(k)$ and stores the key at slot `T[h(k)]` — expected **O(1)** operations.

- **Table size m** = number of slots. **n** = number of stored keys.
- **Load factor α = n / m.**
- **Collision:** two different keys with the same hash value. Collisions are unavoidable (pigeonhole), so we need a *collision-resolution* rule.

Hash functions (what you will meet)

- **Division method:** $h(k) = k \bmod m$ (choose m prime, not close to a power of 2). Almost every GATE question gives you the function explicitly, e.g. $h(k) = k \bmod 10$.
- **Multiplication method:** $h(k) = \lfloor m \cdot (k \cdot A \bmod 1) \rfloor$, $0 < A < 1$.

7.2 Collision resolution method 1 — Chaining (separate chaining)

Each slot holds a **linked list** of all keys that hash there.

- **Insert:** at the head → O(1). **Search/Delete:** walk the list of slot h(k).
- Table never "fills up"; α can exceed 1.
- **Worst case:** all n keys in one chain → search Θ(n).

Example (m = 10, $h(k) = k \bmod 10$, keys 12, 18, 13, 2, 3, 23, 5, 15):

```
slot 2: 12 → 2          slot 3: 13 → 3 → 23     slot 5: 5 → 15       slot 8: 18
(each list shown in insertion order; if inserted at the head the order within a list reverses)
```

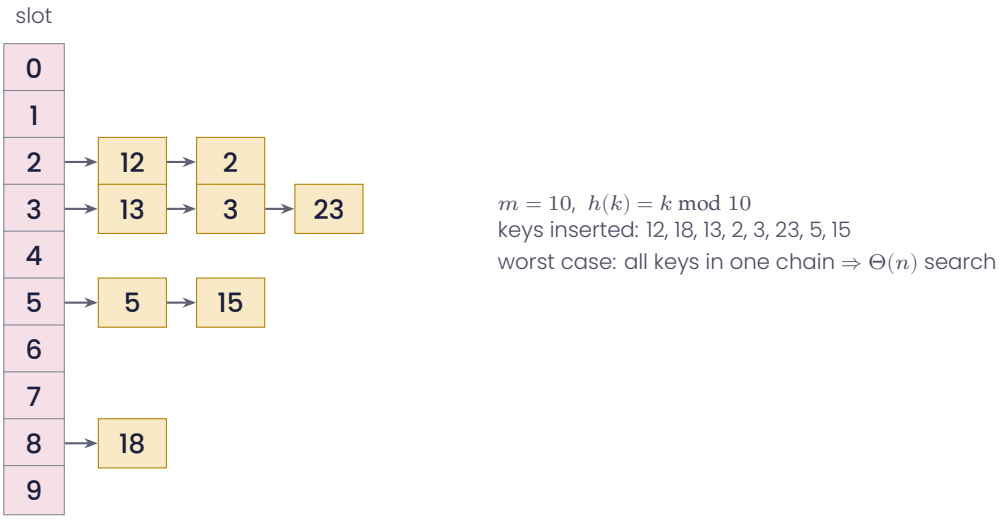


Figure 7.1. Separate chaining: each slot of the table points to a linked list of the keys that hash there.

7.3 Collision resolution method 2 — Open addressing

All keys live **inside the table itself** (so n ≤ m, α ≤ 1). On a collision we **probe** a sequence of slots until a free one is found: `slot(k, i) = ( h(k) + f(i) ) mod m` for i = 0, 1, 2, ...

Scheme	Probe function	Behaviour
Linear probing	$h(k) + i$	try the next slot, then the next... → **primary clustering** (long runs of filled slots grow)
Quadratic probing	$h(k) + c_1 i + c_2 i^2$ (commonly $h(k) + i^2$)	jumps spread out → **secondary clustering** (keys with the same h(k) follow the *same* path); may not visit every slot unless m / c_1, c_2 are chosen well
Double hashing	$h_1(k) + i \cdot h_2(k)$	step size depends on the key → least clustering; needs **$h_2(k) \neq 0$ and coprime with m** (e.g. m prime) to visit all slots

Search follows the same probe sequence and stops at the key (found) or at an **empty** slot (not present). **Deletion** needs care: simply emptying a slot would break other keys' probe paths, so we put a **tombstone / "deleted" marker**. Search skips tombstones; insert may reuse them.

7.4 Fully traced examples (the NAT pattern)

Example 1 — Linear probing. m = 10, h(k) = k mod 10; insert 12, 18, 13, 2, 3, 23, 5, 15 in this order.

Key	h(k)	Probe sequence → final slot	# probes
12	2	2 ✓	1
18	8	8 ✓	1
13	3	3 ✓	1
2	2	2 ✗(12), 3 ✗(13), 4 ✓	3
3	3	3 ✗, 4 ✗(2), 5 ✓	3
23	3	3, 4, 5 ✗, 6 ✓	4
5	5	5 ✗(3), 6 ✗(23), 7 ✓	3
15	5	5, 6, 7, 8 ✗(18), 9 ✓	5

Final table: `[0:-, 1:-, 2:12, 3:13, 4:2, 5:3, 6:23, 7:5, 8:18, 9:15]`; **total probes = 21**, i.e. **13 extra probes** (collision steps).

Example 2 — the three open-addressing schemes side by side. m = 10, insert 89, 18, 49, 58, 69.

- **Linear** (h+i): 89→9; 18→8; 49→9✗→0; 58→8✗,9✗,0✗→**1**; 69→9✗,0✗,1✗→**2**. Table: $0:49, 1:58, 2:69, 8:18, 9:89$.
- **Quadratic** (h+i²): 49→9✗→(9+1)=0; 58→8✗→9✗→(8+4)=12 mod 10 = **2**; 69→9✗→0✗→(9+4)=13 mod 10 = **3**. Table: $0:49, 2:58, 3:69, 8:18, 9:89$.
- **Double hashing** with $h_2(k) = 7 - (k \bmod 7)$: 49→9✗, h₂=7 → (9+7)=16 mod 10 = **6**; 58→8✗, h₂=5 → (8+5)=13 → **3**; 69→9✗, h₂=1 → **0**. Table: $0:69, 3:58, 6:49, 8:18, 9:89$. Notice: linear packs keys into a **cluster** (0,1,2); the others spread out.

Example 3 — m = 7, h(k) = k mod 7, insert 50, 700, 76, 85, 92, 73, 101. h: 50→1, 700→0, 76→6, 85→1, 92→1, 73→3, 101→3.

- *Linear probing:* final `[0:700, 1:50, 2:85, 3:92, 4:73, 5:101, 6:76]` (85 takes 2; 92 tries 1,2→3; 73 tries 3→4; 101 tries 3,4→5).
- *Quadratic* (h+i²): `[700, 50, 85, 73, 101, 92, 76]`.
- *Double hashing,* $h_2(k) = 1 + (k \bmod 5)$: `[700, 50, 85, 73, 92, 101, 76]`. (Same keys, three different layouts — *that is exactly how GATE options differ*: always trace; don't guess.)

Example 4 — tombstone trap. m = 7, linear probing, keys 14, 21, 28 all hash to 0 → stored at 0, 1, 2. **Delete 21** → slot 1 becomes *deleted* (not empty). Searching 28 must **continue past** slot 1 and find it at slot 2. If slot 1 were marked *empty*, "search 28" would wrongly stop and report "not found".



linear probing, $m = 10$: total probes = 21, i.e. 13 extra probes beyond one-per-key

Figure 7.2. Final table after inserting 12, 18, 13, 2, 3, 23, 5, 15 with linear probing: note the cluster in slots 2–9.

7.5 Analysis under the Simple Uniform Hashing Assumption (SUHA)

📌 asked 2022, 2023

SUHA: every key is equally likely to hash to any of the m slots, independently of the others.

Quantity	Value
Probability two **given** distinct keys collide	$\mathbf{1/m}$
Expected number of **keys in a given slot**	$\mathbf{n/m = \alpha}$
Expected number of **other keys colliding with a given key**	$\mathbf{(n-1)/m}$
Expected number of **colliding pairs** (collisions) among n keys	$\mathbf{C(n,2)/m = n(n-1)/(2m)}$
Probability that **n keys occupy distinct slots** (no collision)	**$\prod_{i=0}^{n-1} (1 - i/m)$**
Expected number of **empty slots**	$\mathbf{m(1-1/m)^n}$
Chaining, expected cost of **unsuccessful** search	$\mathbf{\Theta(1+\alpha)}$
Chaining, expected cost of **successful** search	$\mathbf{\approx 1 + \alpha/2}$
Open addressing, expected probes **unsuccessful**	$\mathbf{\leq 1/(1-\alpha)}$
Open addressing, expected probes **successful**	$\mathbf{\leq (1/\alpha)\cdot \ln(1/(1-\alpha))}$

Examples: 10 keys into 20 slots → expected colliding pairs = $C(10,2)/20 = 45/20 = \mathbf{2.25}$. With $\alpha = 0.5$: open addressing needs about **2** probes (unsuccessful) and **1.39** (successful); chaining needs ≈ **1.25** comparisons (successful).

> ⚡ **Memory hooks**
>
> - "Pairs collide with probability $1/m$" ⇒ expected collisions = **number of pairs × $1/m$**. This one line answers most SUHA MCQs.
> - Open addressing blows up as $\alpha \to 1$ ($1/(1-\alpha) \to \infty$); **keep $\alpha < 0.7$** in practice. Chaining degrades gracefully.
> - If *all* keys hash to the same slot (adversarial input) every scheme degrades to $\mathbf{\Theta(n)}$ per operation.

7.6 Chaining vs open addressing

	Chaining	Open addressing
Max load factor	any α (can exceed 1)	$\mathbf{\alpha \leq 1}$
Extra memory	pointers	none
Deletion	easy	needs tombstones
Cache behaviour	worse (pointer chasing)	better
Clustering	none	primary (linear) / secondary (quadratic)

7.7 Practice — Chapter 7

🎯 TRY IT YOURSELF -- cover the answers first

1. **m = 11, h(k)=k mod 11, linear probing. Insert 22, 33, 44. Where is 44?** → 22→0, 33→0✗→1, 44→0✗,1✗→**2**.
2. **Same keys with chaining: length of chain at slot 0?** → **3**.
3. **Expected number of collisions when 20 keys are hashed to 100 slots (SUHA)?** → $C(20,2)/100 = 190/100 = \mathbf{1.9}$.
4. **m = 10, quadratic probing h(k)+i². Insert 19, 29, 39. Where is 39?** → 19→9; 29→9✗→(9+1)=0; 39→9✗,0✗→(9+4)=13 mod 10=**3**.
5. **Linear probing with m = 7, keys 7, 14, 8, 15. Final table?** → 7→0, 14→0✗→1, 8→1✗→2, 15→1✗,2✗→3 ⇒ `[7,14,8,15,-,-,-]`.
6. **Which scheme suffers primary clustering?** → **Linear probing.**
7. **A table with α = 0.9 using open addressing — expected probes for unsuccessful search (upper bound)?** → $1/(1-0.9) = \mathbf{10}$.
8. **Can open addressing store more keys than slots?** → **No** ($n \leq m$).
9. **Why must double-hashing's $h_2(k)$ never be 0?** → the probe would stay on the same slot forever.
10. **Worst-case time to search in chaining with n keys?** → $\Theta(n)$ (all in one chain).

PART

V

Greedy Algorithms and Dynamic Programming

🟡 1.00 markspaper

IN THIS PART

Ch 8 Greedy (Huffman, Scheduling, Knapsack) and Dynamic Programming

CHAPTER 8

Greedy (Huffman, Scheduling, Knapsack) and Dynamic Programming

📝 Notes pp. 99–116. Recall from 5.1: greedy = take the best local choice and never look back; valid when the problem has *optimal substructure* + *greedy-choice property*.

PART A

Greedy Problems

8.1 Huffman Encoding

📝 Lecture notes pp. 99–102

Goal: store text in as few bits as possible. Given how often each character occurs, give frequent characters **short** bit-codes and rare ones **long** codes.

Fixed-length code: every symbol gets the same number of bits (3 bits for ≤ 8 symbols). 📝 p. 99: 225 symbols at 3 bits = **675 bits**. **Variable-length code:** e.g. A = 01, B = 011, C = 1, D = 10, E = 110 → the same data takes **560 bits** — but beware ambiguity: 01101 can be decoded as **A C A** (01·1·01), **A D C** (01·10·1) or **B A** (011·01) ⇒ **ambiguous**.

Prefix-free (prefix) code: *no codeword is a prefix of another.* Then decoding is unambiguous (read bits until you match a codeword; it must be a complete symbol). Every symbol sits at a **leaf** of the code tree. > 📝 p. 100 T/F: *"If we reverse every codeword of a prefix-free code, the result is still prefix-free."* → **False.** Counter-example: {01, 1} is prefix-free; reversed {10, 1} is not (1 is a prefix of 10).

Huffman's algorithm (greedy):

1. Make each character a node with weight = its frequency; put all in a min-priority queue.
2. **Repeat n−1 times:** remove the **two smallest** weights, make them children of a new node whose weight is their **sum**, insert the new node.
3. Label every left edge 0 and right edge 1; a character's code = the path from the root.

Worked example 📝 pp. 100–101 — frequencies (in thousands): **A 45, B 13, C 12, D 16, E 9, F 5** (total 100).

```
Step 1: merge F(5)+E(9)   → 14
Step 2: merge C(12)+B(13) → 25
Step 3: merge 14 + D(16)  → 30
Step 4: merge 25 + 30     → 55
Step 5: merge A(45) + 55  → 100   (root)
```

Codes: **A = 0, B = 101, C = 100, D = 111, E = 1101, F = 1100.** Total bits = 45·1 + 13·3 + 12·3 + 16·3 + 9·4 + 5·4 = 45 + 39 + 36 + 48 + 36 + 20 = **224 bits** (fixed 3-bit code would need 300).

Facts to memorise

- Huffman codes are **prefix-free** (all characters are leaves) and **optimal** among prefix codes.
- The code tree is a **full binary tree**: n leaves, **n−1 internal nodes**, every internal node has 2 children.
- **Time:** n−1 iterations × (2 extract-min + 1 insert = O(log n)) = **O(n log n)**.
- **Cost** = Σ (frequency × code length) = sum of all internal-node weights.
- **Longest possible codeword = n − 1 bits.** Happens with skewed (Fibonacci-like or doubling) frequencies, e.g. 1, 1, 2, 4, 8, 16, 32 (📝 p. 102: each merge creates a node heavier than the next leaf, so the tree is a "spine" of depth n−1 = 6).
- Two lowest-frequency symbols always get the **longest** codes and are **siblings**.
- If all frequencies are equal and $n = 2^k$, Huffman gives a **fixed-length** code of length k.

> ⚡ **Quick cost shortcut:** total encoded length = sum of the weights of **all internal nodes you created** (here 14 + 25 + 30 + 55 + 100 = 224 ✓).

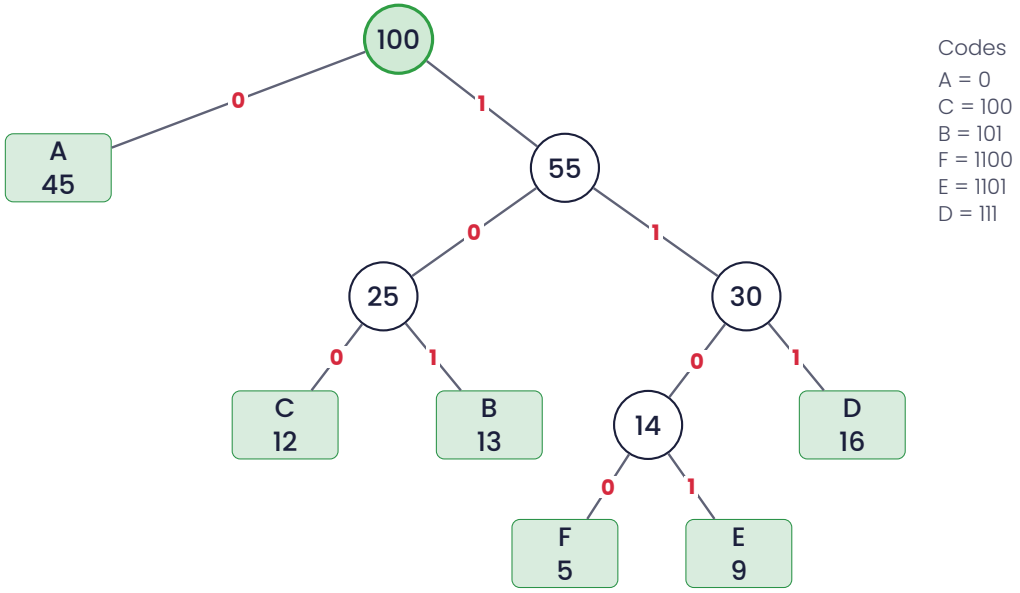


Figure 8.1. Huffman tree for frequencies A:45, B:13, C:12, D:16, E:9, F:5 (merge the two smallest, repeatedly). Frequent symbols get short codes.

8.2 Optimal Merge Pattern

📝 Lecture notes p. 103

Problem: merge n sorted files pairwise into one; merging files of sizes p and q costs p + q record movements. Choose the **merge order** that minimises the total. **Greedy (same as Huffman):** always merge the **two smallest** files.

Example: files **A = 10, B = 20, C = 15, D = 5, E = 25** records.

```
5 + 10 = 15   (D+A)
15 + 15 = 30  (that + C)
20 + 25 = 45  (B+E)
30 + 45 = 75
Record movements = 15 + 30 + 45 + 75 = 165
```

Record comparisons (merging sizes p and q needs $p + q - 1$ comparisons): $14 + 29 + 44 + 74 =$ **161**. > ⚡ *Movements = Σ merged sizes; comparisons = movements − (number of merges).*

8.3 Interval (Activity) Scheduling

📝 Lecture notes pp. 104–105

Problem: from a set of intervals choose the **largest subset with no two overlapping. Greedy:** sort by **finish time**; repeatedly take the interval that **starts at or after the latest finish time** chosen so far.

📝 p. 104 example: [9:00–10:00], [9:30–10:30], [10:00–10:30], [10:15–10:30], [9:45–10:15]. Sorted by finish: [9–10], [9:45–10:15], [9:30–10:30], [10–10:30], [10:15–10:30].

Interval	latest finish	pick?
[9, 10]	0	✓ (finish 10)
[9:45, 10:15]	10	✗ (starts before 10)
[9:30, 10:30]	10	✗
[10, 10:30]	10	✓ (finish 10:30)
[10:15, 10:30]	10:30	✗

Maximum = 2 intervals. Time: sort $O(n \log n)$ + one scan $O(n)$ = $\mathbf{O(n \log n)}$.

- 📝 p. 105: *"Choose the interval that starts last, discard those that conflict, recurse"* — **also correct** (it's the mirror image: process from the right, ordering by latest start time).
- ⚠️ **Wrong greedy rules (classic traps):** earliest *start* time, *shortest* duration, or *fewest conflicts* — each has counter-examples. Only **earliest finish** (or latest start) is correct.

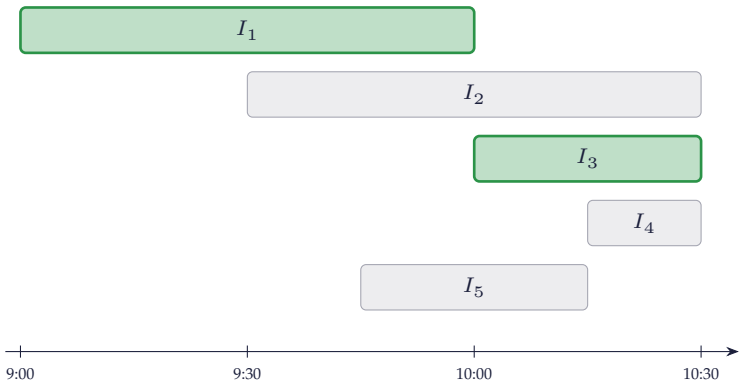


Figure 8.2. Greedy interval scheduling: sort by finish time and keep every interval compatible with the last chosen one (green): I_1 and I_3 are selected.

8.4 Job Sequencing with Deadlines

📝 Lecture notes pp. 105–106

Problem: n jobs, each takes **1 unit of time**, has a **deadline** and a **profit** (earned only if finished by the deadline). Maximise total profit. **Greedy:** sort jobs by **profit, descending**; place each job in the **latest free slot on or before its deadline**; if none is free, discard it.

📝 p. 105 example: (profit, deadline): J0 (10, 2), J1 (30, 7), J2 (15, 5), J3 (20, 4), J4 (40, 3). Order by profit: J4 (40, d3) → slot 3; J1 (30, d7) → slot 7; J3 (20, d4) → slot 4; J2 (15, d5) → slot 5; J0 (10, d2) → slot 2. Schedule: **slot 2 = J0, 3 = J4, 4 = J3, 5 = J2, 7 = J1** → all jobs done, **profit = 115**. **Time:** sorting $O(n \log n)$ + each job scans for a free slot $O(n)$ → $\mathbf{O(n^2)}$ ($O(n \log n)$ with a union-find "next free slot" structure).

8.5 Fractional Knapsack

📝 Lecture notes pp. 106–107

Problem: items with weights and profits; a bag of capacity W; you **may take fractions** of an item. Maximise profit. **Greedy:** sort by **profit/weight ratio, descending**; take whole items while they fit; take a **fraction** of the first one that doesn't.

📝 p. 107 example: capacity **15**. I_1 (w 12, p 4), I_2 (1, 2), I_3 (2, 2), I_4 (1, 1), I_5 (10, 4). Ratios: $I_2 = 2$, $I_3 = 1$, $I_4 = 1$, $I_5 = 0.4$, $I_1 = 0.33$.

Step	Item	Action	Capacity left	Profit
1	I_2	whole	14	2
2	I_3	whole	12	4
3	I_4	whole	11	5
4	I_5	whole	1	9
5	I_1	**1/12** of it	0	$9 + 4/12 = \mathbf{9.33}$

Time $\mathbf{O(n \log n)}$ (the sort). Fractional knapsack **is** solved by greedy; **0/1 knapsack is not** (below).

8.6 Greedy cheat-sheet

Problem	Greedy rule	Time
Activity selection	earliest finish	n log n
Job sequencing (deadlines)	max profit, latest free slot	n^2
Fractional knapsack	highest profit/weight	n log n
Huffman coding	merge two smallest	n log n
Optimal merge	merge two smallest	n log n
MST	Kruskal / Prim	E log V
Single-source shortest path	Dijkstra	(V+E) log V

PART B

Dynamic Programming (dp)

8.7 Why DP?

📝 Lecture notes pp. 109–110

Computing Fibonacci recursively $f(n) = f(n-1) + f(n-2)$ calls the *same* sub-problems again and again (f(n−2) twice, f(n−3) three times …). The call tree has between 2^(n/2) and 2^n nodes ⇒ **O(2^n)**. **Dynamic programming** = *solve each distinct sub-problem only once, store the answer, and reuse it.* Fibonacci becomes **O(n)**.

A problem is a candidate for DP when it has

1. **Overlapping sub-problems** — the recursion revisits the same sub-problems.
2. **Optimal substructure** — an optimal solution is built from optimal solutions of sub-problems.

Two styles: **top-down (memoisation)** — keep the recursion, add a lookup table; **bottom-up (tabulation)** — fill the table in a loop from small to large. Same time complexity.

Recipe to design a DP (use in the exam):

1. Define the state, e.g. `dp[i][j]` = answer for the first i items / first j characters / interval i..j.
2. Write the recurrence (what choices lead to this state?).
3. Base cases.
4. Order of filling.
5. Time = (#states) × (work per state).

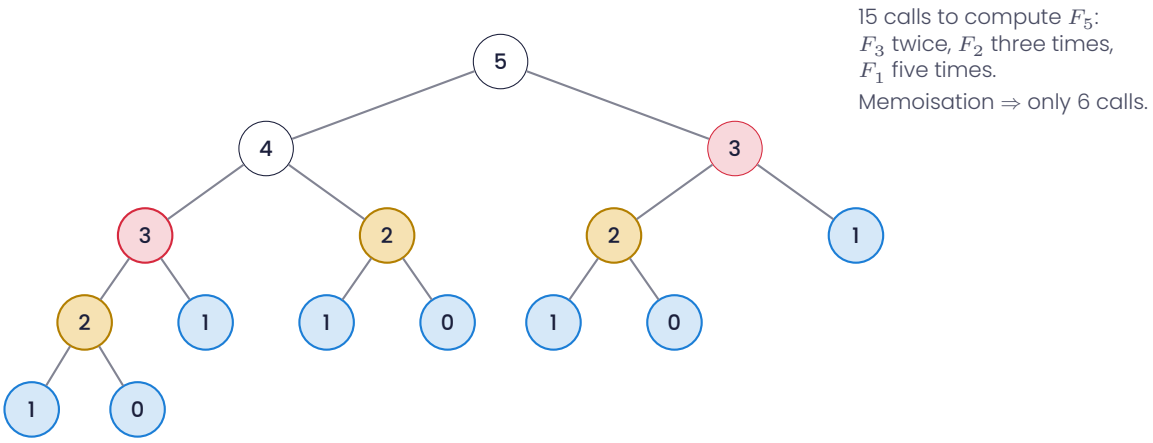


Figure 8.3. Overlapping sub-problems in naive Fibonacci recursion (colours mark repeated work).

8.8 Longest Common Subsequence (LCS)

📝 Lecture notes pp. 111

A **subsequence** is obtained by deleting zero or more characters *without changing the order* ("ACE" is a subsequence of "ABCDE"; it need not be contiguous). **LCS(A, B)** = longest string that is a subsequence of both.

Recurrence (|A| = m, |B| = n; `L[i][j]` = LCS length of the first i characters of A and first j of B):

```
L[i][j] = 0                              if i = 0 or j = 0
        = 1 + L[i-1][j-1]                if A[i] = B[j]
```

```
        = max( L[i-1][j], L[i][j-1] )    if A[i] ≠ B[j]
```

Time: DP **O(m·n)** (m·n cells, O(1) each); space O(mn). Plain recursion: between O(2^min(m,n)) and O(2^(m+n)).

Worked example 📝 p. 111: X = ⟨B, D, C, A, B, A⟩, Y = ⟨A, B, C, B, D, A, B⟩.

```
      ""  A  B  C  B  D  A  B
""     0  0  0  0  0  0  0  0
B      0  0  1  1  1  1  1  1
D      0  0  1  1  1  2  2  2
C      0  0  1  2  2  2  2  2
A      0  1  1  2  2  2  3  3
B      0  1  2  2  3  3  3  4
A      0  1  2  2  3  3  4  4
```

LCS length = 4. Trace back from the bottom-right cell: if letters match go diagonally and record the letter; otherwise move to the larger neighbour. One answer is **BCAB** (also BDAB, BCBA are valid LCSs — *the length is unique, the string isn't*).

> ⚡ **Quick NAT trick:** the answer is the **bottom-right cell**; fill row-by-row: matching letters → diagonal + 1 (copy diag), else max(up, left). **Related:** edit distance has the same shape: `D[i][j] = D[i-1][j-1]` if equal else `1 + min(D[i-1][j], D[i][j-1], D[i-1][j-1])`.

8.9 Matrix Chain Multiplication (MCM)

📝 Lecture notes pp. 112–114

Problem: multiply matrices $A_1A_2\ldots A_n$. Matrix multiplication is associative, so the *order of parenthesisation* doesn't change the result but **hugely changes the cost**. Multiplying a (p×q) matrix by a (q×r) matrix costs **p·q·r** scalar multiplications. Find the order with the **fewest** multiplications.

Why DP: the number of parenthesisations of n matrices is the **Catalan number C(n−1)** = (1/n)·C(2(n−1), n−1): for 4 matrices = (1/4)·C(6,3) = **5**; for 5 matrices = 14; for 6 = 42 — exponential growth.

Example 1 📝 p. 112: A_1 10×100, A_2 100×200, A_3 200×20, A_4 20×10.

Parenthesisation	Cost
$((A_1A_2)A_3)A_4$	10·100·200 + 10·200·20 + 10·20·10 = 200 000 + 40 000 + 2 000 = **242 000**
$(A_1A_2)(A_3A_4)$	200 000 + 40 000 + 20 000 = 260 000
$A_1((A_2A_3)A_4)$	400 000 + 20 000 + 10 000 = 430 000
$(A_1(A_2A_3))A_4$	400 000 + 20 000 + 2 000 = 422 000
$A_1(A_2(A_3A_4))$	40 000 + 200 000 + 10 000 = 250 000

Best = **242 000** — costs differ by almost 2× even for 4 small matrices.

DP formulation: let dimensions array `d[0..n]` (matrix A_i is d[i−1] × d[i]); `M[i][j]` = min cost to multiply $A_i\ldots A_j$.

```
M[i][j] = 0                                                          if i = j
        = min over i ≤ k < j of ( M[i][k] + M[k+1][j] + d[i-1]·d[k]·d[j] )    if i < j
```

(Split the chain after A_k; pay for the left part, the right part, and the final multiplication of the two results.) **Time $O(n^3)$** (n^2 states × up to n splits), **space $O(n^2)$**; naive recursion $\geq O(2^n)$.

Example 2 📝 p. 113: A_1 10×5, A_2 5×20, A_3 20×10, A_4 10×5 ⇒ d = [10, 5, 20, 10, 5].

M	A_1	A_2	A_3	A_4
A_1	0	1000	1500	**1500**
A_2		0	1000	1250
A_3			0	1000
A_4				0

- M[1,2] = 10·5·20 = 1000, M[2,3] = 5·20·10 = 1000, M[3,4] = 20·10·5 = 1000.
- M[1,3] = min(k=1: 0+1000+10·5·10 = 1500 ; k=2: 1000+0+10·20·10 = 3000) = **1500**.
- M[2,4] = min(k=2: 0+1000+5·20·5 = 1500 ; k=3: 1000+0+5·10·5 = 1250) = **1250**.
- M[1,4] = min(k=1: 0+1250+10·5·5 = 1500 ; k=2: 1000+1000+10·20·5 = 3000 ; k=3: 1500+0+10·10·5 = 2000) = **1500** with k = 1. Optimal order: $A_1 \cdot ((A_2A_3)\,A_4)$, cost **1500**.

> ⚡ **Tip:** fill the table diagonal by diagonal (chain length 2, 3, 4…). Number of cells ≈ $n^2/2$, each tries ≤ n−1 splits.

8.10 0/1 Knapsack

📝 Lecture notes pp. 114–115

Problem: n items with profits P_i and weights W_i, a bag of capacity W. For each item you must **take it whole or leave it** (0/1). Maximise profit with total weight $\leq$ W. **Greedy fails** here: items (weight, profit) (10, 60), (20, 100), (30, 120), capacity 50 → greedy by ratio takes the first two (profit 160), but the best is items 2 + 3 = **220**.

DP: `K[i][j]` = best profit using the first i items with capacity j.

```
K[i][j] = 0                                          if i = 0 or j = 0
        = K[i-1][j]                                  if Wᵢ > j        (item doesn't fit)
        = max( K[i-1][j],  Pᵢ + K[i-1][j - Wᵢ] )     if Wᵢ ≤ j        (skip it / take it)
```

Time O(n·W) (this is *pseudo-polynomial* — W is a number's value, not its length); recursion $O(2^n)$.

Worked example 📝 p. 115: items (profit, weight): 1 (2, 1), 2 (2, 2), 3 (3, 4), 4 (5, 5); capacity **6**.

item \ cap	0	1	2	3	4	5	6
none	0	0	0	0	0	0	0
I_1 (2,1)	0	2	2	2	2	2	2
I_2 (2,2)	0	2	2	4	4	4	4
I_3 (3,4)	0	2	2	4	4	5	5
I_4 (5,5)	0	2	2	4	4	5	**7**

Answer **7** (items 1 and 4: weight 1 + 5 = 6, profit 2 + 5). E.g. K[4][6] = max(K[3][6] = 5, 5 + K[3][1] = 5 + 2 = **7**).

8.11 Other DP patterns worth knowing (one-line recurrences)

🆕 ADDED

Problem	State / recurrence	Time
Coin change (min coins)	`c[x] = 1 + min c[x - coin]`	O(n·x) — min coins for 11 with {1,2,5} = **3** (5+5+1)
Rod cutting	`r[n] = max_{1≤i≤n} (price[i] + r[n-i])`	O(n²) — prices 1,5,8,9,10,17,17,20 and length 8 → **22** (cut 2 + 6)
Longest Increasing Subsequence	`L[i] = 1 + max L[j], j < i, A[j] < A[i]`	O(n²) (O(n log n) with binary search) — [10,9,2,5,3,7,101,18] → **4**
Max subarray (Kadane)	`best_ending_here = max(A[i], best_ending_here + A[i])`	**O(n)**
Edit distance	see 8.8	O(mn) — kitten → sitting = **3**
Floyd–Warshall (Ch 5)	`d_k[i][j]`	O(V³)
Bellman–Ford (Ch 5)	`d_k[v]` = best with ≤ k edges	O(VE)

8.12 Greedy vs Dynamic Programming — decision rule

- If a **local best choice is provably safe** (Huffman, MST, activity selection, fractional knapsack) → **greedy** (fast, simple).
- If choices **interact** and you must compare combinations (0/1 knapsack, LCS, matrix chain) → **DP**.
- If neither structure exists → brute force / backtracking (NP-hard territory).

8.13 Practice — Chapter 8

🎯 TRY IT YOURSELF -- cover the answers first

1. **Huffman for frequencies 5, 9, 12, 13, 16, 45. Total encoded bits (weights sum of internal nodes)?** → 14 + 25 + 30 + 55 + 100 = **224**.
2. **Longest Huffman codeword for n = 8 symbols?** → at most **7**.
3. **Optimal merge of files 2, 3, 4, 5 — total cost?** → 2+3 = 5; 4+5 = 9; 5+9 = 14 → **28**.
4. **Fractional knapsack: capacity 10, items (w, p) = (5, 30), (4, 20), (6, 12).** Ratios 6, 5, 2 → take (5,30) and (4,20) = 50, then 1/6 of the third (profit 2) = **52**.
5. **LCS length of "ABCBDAB" and "BDCABA"** → **4**.
6. **Number of ways to parenthesise a product of 5 matrices?** → Catalan C_4 = **14**.
7. **MCM time and space?** → **O(n³)** and **O(n²)**.

8. **0/1 knapsack, n = 100 items, W = 10^6. DP time?** → **$O(nW) = 10^8$** operations.
9. **Interval scheduling: [1,4], [3,5], [0,6], [5,7], [3,9], [5,9], [6,10], [8,11], [8,12], [2,14], [12,16] — max activities?** → 4 ([1,4], [5,7], [8,11], [12,16]).
10. **True/False:** (a) Greedy solves 0/1 knapsack ✗ (b) DP solves fractional knapsack better than greedy ✗ (greedy is optimal and faster) (c) Huffman codes are prefix-free ✓ (d) LCS of two strings is unique ✗ (only its length is).

PART

VI

Recursion, Heaps, Trees, Searching



IN THIS PART

Ch 9 Recursion and Code Tracing

Ch 10 Heaps, Binary Search Trees and Searching

CHAPTER 9

Recursion and Code Tracing

1.06 marks/paper; 7 questions in 2018–2026

🆕 NEW • not in your lecture notes

> Not in your notes, but the PYQ analysis shows **"trace a recursive function / what does this compute?"** in 2018, 2019, 2020, 2021 (×3) and 2026. These are NAT questions: you must produce the exact number.

9.1 How recursion works (from zero)

A recursive function calls itself on a smaller input until a **base case** stops it. Each call gets its own **stack frame** (own copy of the parameters/local variables). The calls pile up (call stack) and then unwind, returning values back up.

- **Space** used = **depth of the recursion** (× frame size). Fib(n) has depth n ⇒ O(n) space even though time is exponential.
- **Time** = (number of calls) × (work per call) → write a recurrence (Ch 2).

9.2 The 5-step tracing method

1. **Find the base case(s)** and their return values.
2. **Try tiny inputs** (n = 0, 1, 2, 3) by hand and write them in a table.
3. **Spot the pattern** (is it Fibonacci? halving? digit-by-digit? powers of 2?).
4. **Confirm** with one more value, then plug in the asked input.
5. For *"how many times is X executed"* questions write a **recurrence for the count** and solve (Ch 2).

> ⚠️ **Watch for: global/static variables** that persist across calls; **order of printing** (before vs after the recursive call); **integer division** (n/2 truncates); **two recursive calls in one expression** (both are evaluated); **pass-by-value vs pointer** parameters.

9.3 Pattern library (with verified answers)

Pattern	Code idea	Example result
Fibonacci-like $f(n)=f(n-1)+f(n-2)$ (base f(0)=f(1)=1)	two calls, depth n	values 1,1,2,3,5,8,13; **number of calls** C(n)=C(n−1)+C(n−2)+1: C(4)=9, **C(5)=15**, C(6)=25
Tribonacci-like $h(n)=h(n-1)+h(n-2)+h(n-3)$, h(n≤0)=1		1, 3, 5, 9, 17, 31, 57, 105
Binary digits / popcount `f(n)= n==0 ? 0 : n%2 + f(n/2)`	counts 1-bits	f(37)=3 (100101), f(255)=8
Digit sum `f(n)= n==0 ? 0 : n%10 + f(n/10)`		f(4821) = 15
Halving count $f(n)=n\le 1?0:1+f(n/2)$	$\lfloor \log_2 n \rfloor$	f(1000)=9, f(1024)=10
Euclid `gcd(a,b)= b==0 ? a : gcd(b, a%b)`		gcd(48,18): (48,18)→(18,12)→(12,6)→(6,0) = **4 calls**, result 6
Tower of Hanoi $H(n)=2H(n-1)+1$		H(5) = **31** moves (2^n-1)
In-order of two calls `if(n>0){ f(n-1); print; f(n-1); }`	prints 2^n-1 times	n=3 → 7 prints
Loop + recursion `for i<n {…} ; f(n-1)`	n + (n−1) + … + 1	g(5) = **15** = n(n+1)/2, $\Theta(n^2)$
Ackermann A(m,n)	nested recursion	A(2,3) = **9**, A(1,5) = 7
McCarthy-91 $f(n)=n>100?n-10:f(f(n+11))$	nested	f(95) = f(50) = **91** (for all n ≤ 100)

9.4 Worked examples

Ex 1 (call order). `f(n){ if(n>0){ print("a",n); f(n-1); print("b",n); f(n-1);} }`, call f(2). Trace: a2 → f(1): a1, f(0), b1, f(0) → back: b2 → f(1): a1, b1. Output: **a2 a1 b1 b2 a1 b1**. (Preorder-style prints come before the recursive call, postorder-style after.)

Ex 2 (fast power). `pw(x,n){ if(n==0) return 1; t=pw(x,n/2); return n even ? t*t : t*t*x; }` pw(2,10) → pw(2,5) → pw(2,2) → pw(2,1) → pw(2,0)=1 ⇒ 2, 4, 32, 1024. **5 calls in total ($\lfloor \log_2 n \rfloor + 2$)**; multiplications ≤ $2\lfloor \log_2 n \rfloor$.

Ex 3 (how many calls). `f(n)= n<=1 ? 1 : f(n-1)+f(n-2)`: calls C(n)=C(n−1)+C(n−2)+1 ⇒ f(5) makes **15** calls and returns 8.

Ex 4 (nested). A(2,3): A(2,n)=2n+3 ⇒ A(2,3)=**9**. (A(0,n)=n+1; A(1,n)=n+2; A(2,n)=2n+3.)

9.5 Time & space of recursive code — fast verdicts

Code shape	Time	Space (stack)
single call on n−1, O(1) work	$\Theta(n)$	$\Theta(n)$
single call on n/2, O(1) work	$\Theta(\log n)$	$\Theta(\log n)$
two calls on n−1	$\Theta(2^n)$	$\Theta(n)$
two calls on n/2 + O(n) work	$\Theta(n \log n)$	$\Theta(\log n)$
memoised (DP) recursion	Θ(#states × work)	Θ(#states)

9.6 Practice — Chapter 9

🎯 TRY IT YOURSELF -- cover the answers first

1. `f(n)= n<=1 ? 1 : f(n-1)+f(n-2)`. **How many calls for f(6)? → 25.**
2. `f(n)= n==0 ? 0 : n%2 + f(n/2)`; **f(100)?** → $100 = 1100100_2$ → **3.**
3. **gcd(1071, 462) — number of recursive calls?** → $(1071,462) \to (462,147) \to (147,21) \to (21,0)$: **4 calls**, gcd 21.
4. **Hanoi with 6 discs: moves? → 63.**
5. **Time of** $g(n) = g(n-1) + n$ **loops?** → $\Theta(n^2)$.
6. **Space used by recursive binary search?** → $\Theta(\log n)$.
7. $h(n) = h(n-1) + h(n-2) + h(n-3)$**, base 1 for n ≤ 0: h(5)? → 31.**
8. **McCarthy-91: f(87)? → 91.**

CHAPTER 10

Heaps, Binary Search Trees and Searching

skim, — but 2026 had 4 marks here

NEW • not in your lecture notes

10

10.1 Binary heap

A **binary heap** is a **complete binary tree** stored in an **array**, with the **heap property**:

- **Max-heap:** every parent ≥ its children (the root is the maximum).
- **Min-heap:** every parent ≤ its children (the root is the minimum).

Array layout (1-indexed): root at 1; node i has **parent ⌊i/2⌋, left child 2i, right child 2i+1**. (0-indexed: parent ⌊(i−1)/2⌋, children 2i+1, 2i+2.) **Height** of an n-node heap = **⌊log₂ n⌋**; number of **leaves = ⌈n/2⌉** (they occupy positions ⌊n/2⌋+1 ... n).

Operation	How	Time
Find max (min-heap: min)	root	**O(1)**
Insert	put at the end, **sift up**	O(log n)
Extract-max	swap root with last, remove, **sift down** (Heapify)	O(log n)
Build-heap from n unsorted items	call Heapify on nodes ⌊n/2⌋ ... 1 (bottom-up)	**Θ(n)** (not n log n!)
Heap sort	build-heap, then n times extract-max	**Θ(n log n)**, in-place, unstable
Find **min in a max-heap**	scan the leaves	Θ(n)
Search arbitrary key	no order between siblings	Θ(n)

Why build-heap is Θ(n): a node at height h costs O(h), and there are ≤ n/2^(h+1) such nodes: Σ h·n/2^(h+1) = O(n).

Worked build-heap (max-heap): A = [4, 10, 3, 5, 1]. Heapify node 2 (value 10: children 5, 1 — fine). Heapify the root 4: larger child is 10 → swap → [10, 4, 3, 5, 1]; 4 now has children 5, 1 → swap with 5 → **[10, 5, 3, 4, 1]**.

Heap facts for MSQs

- The **2nd largest** in a max-heap is a **child of the root**.
- The **smallest** element of a max-heap is always at a **leaf**.
- Sorted array (ascending) is a valid **min-heap**.
- **k-th smallest** via a min-heap: build Θ(n) + k extract-mins Θ(k log n).

- **Merging K sorted lists** with a heap: **$\Theta(n \log K)$** (Ch 3).
- **Sorting a k-sorted array** (each element $\le$ k positions from its place): min-heap of size k+1 → **$\Theta(n \log k)$**.

Counting distinct heaps (a favourite NAT) 🆕: with n **distinct** keys the number of different max-heaps is $H(n) = C(n-1, L) \cdot H(L) \cdot H(R)$, where L and R are the sizes of the left and right subtrees of the *complete-tree shape* (the biggest element must be the root; choose which L of the remaining n−1 go left). Values: H(1)=1, H(2)=1, H(3)=**2**, H(4)=**3**, H(5)=**8**, H(6)=**20**, H(7)=**80**, H(8)=210, H(9)=896, H(10)=3360, H(15)=21 964 800.

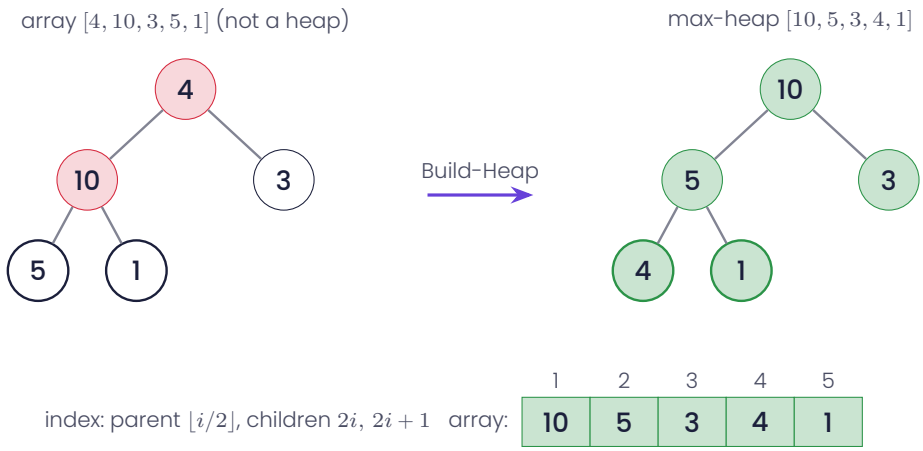
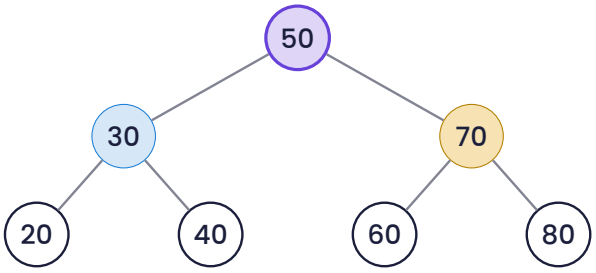


Figure 10.1. Building a max-heap bottom-up in $\Theta(n)$ time; the tree is stored in a plain array.

10.2 Binary Search Tree (BST)

BST property: for every node, all keys in its **left** subtree are **smaller**, all keys in its **right** subtree are **larger**.

- **In-order traversal** (left, node, right) outputs the keys **in sorted order**.
- **Search / insert / delete** take $\mathbf{O(h)}$ where h = height: best $\lfloor \log_2 n \rfloor$ (balanced), worst $n - 1$ (a chain — e.g. inserting already-sorted keys).
- **Deletion:** leaf → remove; one child → splice; two children → replace by the **in-order successor** (smallest in right subtree) or predecessor, then delete that node.
- **Number of different BSTs with n distinct keys = Catalan $C_n = C(2n, n)/(n+1)$**: n=3 → **5**, n=4 → 14, n=5 → 42. (Number of *different insertion orders* producing a given shape varies.)
- **Rebuilding from a traversal:** the **preorder** (or postorder) of a BST uniquely determines it (the first preorder element is the root; split the rest at the first larger key). *Example:* preorder **50, 30, 20, 40, 70, 60, 80** → root 50, left {30,20,40}, right {70,60,80} → **postorder: 20, 40, 30, 60, 80, 70, 50**; inorder: 20 30 40 50 60 70 80.
- **Self-balancing (AVL, red-black):** guarantee h = O(log n). **AVL** minimum nodes for height h: N(0)=1, N(1)=2, N(2)=4, N(3)=7, N(h)=N(h−1)+N(h−2)+1 ⇒ height ≤ 1.44·log₂ n.



Pre-order (root, L, R): 50, 30, 20, 40, 70, 60, 80
In-order (L, root, R): 20, 30, 40, 50, 60, 70, 80 (sorted!)
Post-order (L, R, root): 20, 40, 30, 60, 80, 70, 50

Figure 10.2. A BST built from the pre-order 50, 30, 20, 40, 70, 60, 80; in-order traversal always yields the keys in sorted order.

10.3 Searching

Method	Needs	Time
Linear search	nothing	O(n)
Binary search	sorted array	**O(log n)** (Ch 3)
Interpolation search	sorted, uniform	O(log log n) average
Search in a bitonic array	increasing then decreasing	find the peak with binary search (O(log n)), then binary-search each side → **O(log n)**
Search in a rotated sorted array	sorted then rotated	**O(log n)** (find the pivot)
Search in a row-wise & column-wise sorted n×m matrix		**O(n + m)** (staircase walk from a corner)
Hash table	hash function	O(1) expected (Ch 7)

> ⚡**Rules of thumb:** any structure that lets you *discard half the candidates each step* gives $O(\log n)$. If a problem says "sorted" and the expected answer is $\Theta(n)$, look for the binary-search idea.

10.4 Practice — Chapter 10

🎯 TRY IT YOURSELF -- cover the answers first

1. **Height of a heap with 1000 nodes?** → $\lfloor \log_2 1000 \rfloor = \mathbf{9}$.
2. **Build a max-heap by inserting 1,2,3,4,5 one at a time vs build-heap on [1,2,3,4,5]?** → insert-one-by-one: $\Theta(n \log n)$ worst; build-heap $\Theta(n)$.
3. **Distinct max-heaps of 5 distinct keys?** → **8**.
4. **Number of BSTs on 4 keys?** → **14**.
5. **Min nodes in an AVL tree of height 3 (single node = height 0)?** → **7**.
6. **Where can the minimum of a max-heap of 15 elements be?** → one of the **8 leaves** (positions 8–15).
7. **Worst-case search time in a BST built from sorted input?** → $\boldsymbol{\Theta(n)}$.
8. **Time to merge 10 sorted lists with n total elements using a heap?** → $\Theta(n \log 10) = \boldsymbol{\Theta(n)}$ for constant K ($\Theta(n \log K)$ in general).

PART

VII

Revision Tools

IN THIS PART

Ch 11 One-Page Cheat Sheets

Ch 12 PYQ Practice Map — Solve These Yourself

CHAPTER 11

One-Page Cheat Sheets

11.1 Complexity master table

Algorithm / task	Time	Space	Notes
Binary search	$\Theta(\log n)$	$O(1)$ iter / $O(\log n)$ rec	sorted input
Merge sort	$\Theta(n \log n)$ all cases	$\Theta(n)$	stable
Quick sort	best/avg $\Theta(n \log n)$, worst $\mathbf{\Theta(n^2)}$	$O(\log n)$	in-place, unstable
Heap sort	$\Theta(n \log n)$	$O(1)$	in-place, unstable
Insertion / Bubble	best $\Theta(n)$, worst $\Theta(n^2)$	$O(1)$	stable
Selection sort	$\Theta(n^2)$ always, $\leq n-1$ swaps	$O(1)$	unstable
Counting sort / Radix	$\Theta(n+k)$ / $\Theta(d(n+k))$	$\Theta(n+k)$	stable, non-comparison
Quick-select (k-th smallest)	avg $\Theta(n)$, worst $\Theta(n^2)$		median-of-medians → worst $\Theta(n)$
Build heap	$\mathbf{\Theta(n)}$		insert/extract $\Theta(\log n)$
Max & min together	$\lceil 3n/2 \rceil - 2$ comparisons		naive $2n-2$
Max & 2nd max	$n + \lceil \log n \rceil - 2$		tournament
Count inversions	$\Theta(n \log n)$		merge-sort based
Closest pair (2-D)	$\Theta(n \log n)$		D&C
Fast power a^b	$\Theta(\log b)$		
Strassen	$\Theta(n^{\wedge}2.81)$		naive $\Theta(n^3)$
BFS / DFS	$\Theta(V+E)$ (AL), $\Theta(V^2)$ (AM)	$O(V)$	
Topological sort	$\Theta(V+E)$		DFS finish order reversed / Kahn
SCC (Kosaraju)	$\Theta(V+E)$		two DFS passes
Dijkstra	$O((V+E) \log V)$ heap+AL; $O(V^2)$ array; $O(V \log V + E)$ Fibonacci		no negative edges
Bellman–Ford	$O(VE)$		negative edges, detects negative cycles
DAG shortest path	$\Theta(V+E)$		negatives OK

Algorithm / task	Time	Space	Notes
Floyd–Warshall	$\Theta(V^3)$	$\Theta(V^2)$	all pairs
Kruskal	$\Theta(E \log E) = \Theta(E \log V)$		Union-Find
Prim	$O((V+E) \log V)$ heap; $O(V^2)$ array; $O(E + V \log V)$ Fibonacci		
Huffman	$\Theta(n \log n)$		$n-1$ merges
Fractional knapsack	$\Theta(n \log n)$		greedy
Job sequencing	$\Theta(n^2)$		greedy by profit
Interval scheduling	$\Theta(n \log n)$		earliest finish
LCS	$\Theta(mn)$	$\Theta(mn)$	
Matrix chain	$\Theta(n^3)$	$\Theta(n^2)$	
0/1 knapsack	$\Theta(nW)$	$\Theta(nW)$	pseudo-polynomial
TSP brute force / DP	$\Theta((n-1)!)$ / $\Theta(n^2 2^n)$		NP-hard
Hashing (SUHA)	$\Theta(1+\alpha)$ expected		worst $\Theta(n)$

11.2 Recurrence one-liners

Recurrence	Θ
$T(n-1)+1$	n
$T(n-1)+n$	n^2
$T(n-1)+\log n$	$n \log n$
$2T(n-1)+1$	2^n
$T(n-1)+T(n-2)+1$	φ^n (between 2^(n/2) and 2^n)
$T(n/2)+1$	$\log n$
$T(n/2)+n$	n
$2T(n/2)+1$	n
$2T(n/2)+n$	$n \log n$
$2T(n/2)+n \log n$	$n \log^2 n$
$2T(n/2)+n/\log n$	$n \log \log n$
$2T(n/2)+n^2$	n^2
$4T(n/2)+n^2$	$n^2 \log n$
$8T(n/2)+n^2$	n^3
$7T(n/2)+n^2$	n^2.81
$T(n/3)+T(2n/3)+n$	$n \log n$
$T(\sqrt{n})+1$	$\log \log n$
$T(\sqrt{n})+\log n$	$\log n$
$2T(\sqrt{n})+\log n$	$\log n \cdot \log \log n$

11.3 Graph algorithm chooser

If the question says…	Think…
"cycle", "back edge", "DAG"	DFS timestamps / edge classification
"order of tasks", "prerequisites"	topological sort
"fewest hops", "unweighted shortest"	BFS
"bipartite", "2-colourable", "odd cycle"	BFS levels
"cut vertex", "bridge", "critical"	DFS low-link
"strongly connected", "mutually reachable"	SCC (Kosaraju)
"cheapest route from s" (≥ 0 weights)	Dijkstra
"negative weights", "negative cycle"	Bellman–Ford
"all pairs"	Floyd–Warshall
"connect all at min total cost"	MST (Kruskal/Prim)
"number of MSTs", "weight of MST"	cut/cycle property, equal-weight groups

11.4 Top 40 facts to know cold

1. O is ceiling, Ω floor, Θ both. 2. $2^{n+1} = \Theta(2^n)$, $2^{2n} \neq O(2^n)$. 3. $\log(n!) = \Theta(n \log n)$. 4. Loop $j+ = i$ inside `for i` → n log n. 5. Master: compare n^(log_b a) with f(n); log factors need the extended table. 6. $a = b^k, p = -1 \rightarrow n \log\log n$. 7. Σαᵢ rule for unequal splits. 8. $T(\sqrt{n})$ → substitute n = 2^m.
2. Merge sort Θ(n log n) always; stable; Θ(n) space. 10. Quick sort worst Θ(n²). 11. Comparison sorts need Ω(n log n). 12. Insertion-sort swaps = inversions. 13. Max inversions n(n−1)/2. 14. Pair method 3n/2−2. 15. 2nd max n+log n−2.
3. DFS/BFS Θ(V+E). 17. Undirected DFS: only tree + back edges. 18. Cycle ⇔ back edge. 19. DAG edge u→v ⇒ f[u] > f[v]. 20. Root is AP iff ≥ 2 children; non-root: child with low ≥ disc. 21. BFS same-level edge ⇒ odd cycle ⇒ not bipartite. 22. Tree that is both DFS and BFS tree ⇒ G = T.
4. Dijkstra fails on negative edges; needs ≥ 0. 24. Bellman–Ford V−1 passes + 1 detection pass. 25. DAG shortest path Θ(V+E). 26. Adding a constant to all edges breaks shortest paths, not MST.
5. Spanning tree has V−1 edges; K_n has n^{n-2}. 28. Cut property / cycle property. 29. Distinct weights ⇒ unique MST. 30. Kruskal sorts edges; works with negatives; squaring weights keeps MST. 31. Prim ≠ Dijkstra. 32. Count MSTs = product over equal-weight groups.
6. Hashing: load factor α = n/m; expected colliding pairs C(n,2)/m. 34. Linear probing → primary clustering. 35. Tombstones for deletion.
7. LCS table: match → diagonal+1; else max(up,left). 37. MCM: M[i][j]=min(M[i][k]+M[k+1][j]+d[i−1]d[k]d[j]). 38. 0/1 knapsack DP O(nW); fractional knapsack greedy. 39. Huffman: merge two smallest; cost = sum of internal nodes; max code length n−1. 40. Build-heap is Θ(n).

11.5 Exam-day strategy

- **1-mark recurrences/orderings first** — they are the safest marks.
- **NAT:** write the intermediate table (Dijkstra, Bellman–Ford, hash, DP) — don't compute in your head.
- **MSQ:** judge each option independently; a *true but loose* bound (like $n = O(n^2)$) is still true.
- **Graph question with a figure:** write adjacency lists first; fix a visiting order (alphabetical/numeric unless told otherwise).
- **Time-box:** if a counting-MST or tracing question passes 4 minutes, mark it and move on.

CHAPTER 12

PYQ Practice Map — Solve These Yourself

12

Your analysis PDF (Appendix A.5) lists **77 Algorithms questions from 2018 to 2026**. The table below sorts them by cluster so that after each chapter you can solve exactly the right ones. **Format:** `year / set, Q-number (marks, type)`. Find each question in the analysis PDF / GATE paper and attempt it *without* looking at this book, then compare your reasoning with the chapter.

Cluster	Study in	# Qs	Questions to solve (year / set, Q#, marks, type)
Recurrences	Ch 2	8	2026 S1 Q7 (1M,MCQ); 2026 S2 Q15 (1M,MSQ); 2025 S1 Q10 (1M,MCQ); 2024 S1 Q32 (2M,MCQ); 2024 S2 Q5 (1M,MCQ); 2021 S1 Q30 (2M,MCQ); 2021 S2 Q39 (2M,MCQ); 2020 Q2 (1M,MCQ)
Growth rates	Ch 1	4	2026 S2 Q14 (1M,MCQ); 2023 Q19 (1M,MSQ); 2022 Q1 (1M,MCQ); 2021 S1 Q3 (1M,MCQ)
Code complexity	Ch 1	4	2026 S2 Q28 (2M,MCQ); 2024 S1 Q7 (1M,MCQ); 2023 Q44 (2M,MSQ); 2019 Q37 (2M,MCQ)
BFS/DFS	Ch 4	7	2026 S1 Q40 (2M,MSQ); 2025 S1 Q33 (2M,MCQ); 2025 S2 Q49 (2M,NAT); 2024 S1 Q35 (2M,MSQ); 2024 S1 Q50 (2M,NAT); 2023 Q46 (2M,NAT); 2018 Q30 (2M,MCQ)
DAG/SCC/Topo	Ch 4	4	2026 S2 Q27 (2M,MCQ); 2021 S2 Q46 (2M,MSQ); 2021 S2 Q55 (2M,NAT); 2018 Q43 (2M,NAT)
Shortest paths	Ch 5	4	2026 S1 Q31 (2M,MCQ); 2025 S1 Q8 (1M,MCQ); 2025 S2 Q27 (2M,MCQ); 2020 Q40 (2M,MCQ)
MST counting	Ch 6	6	2025 S1 Q54 (2M,NAT); 2024 S2 Q49 (2M,NAT); 2022 Q48 (2M,NAT); 2021 S1 Q17 (1M,NAT); 2020 Q49 (2M,NAT); 2018 Q47 (2M,NAT)
MST properties	Ch 6	4	2026 S1 Q39 (2M,MSQ); 2022 Q39 (2M,MSQ); 2021 S2 Q1 (1M,MCQ); 2020 Q31 (2M,MCQ)
Hashing	Ch 7	5	2026 S1 Q14 (1M,MSQ); 2026 S2 Q20 (1M,NAT); 2025 S1 Q55 (2M,NAT); 2021 S1 Q47 (2M,MSQ); 2020 Q23 (1M,NAT)
Hashing (SUHA)	Ch 7	2	2023 Q10 (1M,MCQ); 2022 Q6 (1M,MCQ)
Recursion tracing	Ch 9	7	2026 S1 Q51 (2M,NAT); 2021 S1 Q48 (2M,NAT); 2021 S2 Q23 (1M,NAT); 2021 S2 Q49 (2M,NAT); 2020 Q48 (2M,NAT); 2019 Q26 (2M,MCQ); 2018 Q45 (2M,NAT)

Cluster	Study in	# Qs	Questions to solve (year / set, Q#, marks, type)
DP	Ch 8	5	2026 S2 Q29 (2M,MCQ); 2024 S2 Q32 (2M,MCQ); 2021 S1 Q40 (2M,MSQ); 2019 Q25 (1M,NAT); 2018 Q31 (2M,MCQ)
Greedy/Huffman	Ch 8	2	2021 S2 Q26 (2M,MCQ); 2018 Q48 (2M,NAT)
Sorting trace	Ch 3	4	2026 S2 Q22 (1M,NAT); 2025 S1 Q23 (1M,NAT); 2024 S1 Q31 (2M,MCQ); 2019 Q20 (1M,NAT)
Sorting theory	Ch 3	3	2025 S2 Q10 (1M,MCQ); 2024 S2 Q25 (1M,NAT); 2021 S1 Q9 (1M,MCQ)
Searching	Ch 3 / 10	3	2025 S2 Q19 (1M,MSQ); 2025 S2 Q31 (2M,MCQ); 2021 S2 Q8 (1M,MCQ)
BST/Trees	Ch 10	4	2026 S1 Q23 (1M,NAT); 2026 S1 Q30 (2M,MCQ); 2026 S1 Q52 (2M,NAT); 2026 S2 Q39 (2M,MSQ)
Heap	Ch 10	1	2026 S1 Q13 (1M,MSQ)

12.1 Suggested sequence for PYQ practice

1. **After Ch 1–2:** all *Recurrences*, *Growth rates*, *Code complexity* questions (16 Qs).
2. **After Ch 4–6:** *BFS/DFS*, *DAG/SCC*, *Shortest paths*, *MST counting*, *MST properties* (25 Qs) — highest return per hour.
3. **After Ch 7, 3, 8–10:** hashing, sorting, DP/greedy, tracing, trees/heaps.
4. **Last 3 days:** re-attempt every question you got wrong; speed-drill the NATs.

12.2 Pattern → what to do (from the cluster analysis)

Pattern asked in PYQs	Your weapon
"Solve the recurrence / which is the tight bound?"	Ch 2 flow-chart: Master, extended Master, tree, substitution
"Arrange functions in order of growth"	Ch 1.4 ladder + log-comparison trick
"Time complexity of this code fragment"	Ch 1.6 loop table
"Which statements about BFS/DFS/DAG are true?"	Ch 4 tables (edge types, timestamps, level rules)
"Run Dijkstra / Bellman–Ford, give the value/condition"	Ch 5 tables; replay-the-history trick (5.3.1)
"Number or weight of MSTs"	Ch 6.6 method (equal-weight groups), cut/cycle properties
"Insert keys into a hash table, find slot / collisions"	Ch 7 traced examples; SUHA table
"What does this recursive function return?"	Ch 9 pattern library
"Which DP recurrence / how many entries / Huffman length"	Ch 8 recurrences and tables

Appendix A: What came from your notes vs what I added

Fully covered from your lecture notes (all examples included): asymptotics intro · repeated substitution · recursion tree · Master theorem (basic + complete) · variable-change recurrences · D&C: max, merge sort (all variants & iterative), merging K arrays, stable/in-place, max/min/2nd-max, inversions, closest pair, fast power, matrix multiplication, quick sort, quick-select, binary search · graph representations · DFS (timestamps, parenthesis theorem, edge classification, cycle, topological sort, articulation points) · BFS (layers, edges, bipartite, components) · greedy basics · Dijkstra (incl. negative edges, complexity table) · DAG shortest path · Bellman–Ford · MST (cut/cycle property, Kruskal, Prim, counting, range-of-x, updates) · Huffman · optimal merge · interval scheduling · job sequencing · fractional knapsack · DP (LCS, matrix chain, 0/1 knapsack) · Floyd–Warshall, TSP · bubble/insertion/selection sort · comparison lower bound.

🆕 **Added because the PYQ analysis weights it (not in the notes):** growth-rate ladder and loop-counting tables (Ch 1) · Strassen · median-of-medians clarification · low-link formula for AP/bridges · SCC/Kosaraju · white-path theorem · complete hashing chapter (Ch 7) · recursion-tracing chapter (Ch 9) · heaps / BST / searching (Ch 10) · other DP patterns · cheat sheets and PYQ map.

Corrections I made to the handwritten notes (so you don't memorise a typo):

1. 3-way merge: worst-case comparisons = $\mathbf{5n/3 - 2}$ (notes show $n - 2$).
2. $T(n/2)+T(n/3)+T(n/4)+1$ upper bound exponent = $\mathbf{\log_2 3 \approx 1.58}$ (notes show 1.5).
3. Extended Master, $a < b^k$ and $p < 0 \rightarrow \mathbf{\Theta(n^k)}$ (notes write $O(n^k)$; Θ is the tight statement).
4. Four-blob MST count = $3^2 \times 4^2 = \mathbf{144}$ (notes show 149).
5. Median-of-medians (groups of 5) gives **worst-case $\mathbf{\Theta(n)}$**; the plain random-pivot quick-select is the one with worst case $\Theta(n^2)$.

Final word



The scoring recipe, in order: **recurrences → graph traversals/shortest paths → MST counting → hashing tracing → everything else.** Do every worked example by hand once, every practice set once, every PYQ once, and re-read Chapter 11 the night before. Good luck! 🚀